

Deep Learning

Session 5: Computer vision (2): Modern CNNs,
implementation, and applications

October 1, 2024

This session will be recorded for in-class use.

Deep learning theory

1. Deep learning in public policy
2. Deep neural networks (1)
3. Deep neural networks (2)

Applications

4. Computer vision (1): Convolutional neural networks
5. **Computer vision (2): Modern CNNs, implementation, and applications**
6. Sequence methods and time-series analysis
7. Natural language processing (1)
8. Natural language processing (2): Transformer models, implementation, and applications
- Midterm exam -
9. Further topics in deep learning

Deep learning and public policy

10. Policy approaches to regulate deep learning and responsible implementation in government
11. Deep learning in practice
12. Tutorial presentations

Computer vision (2): Modern CNNs, implementation, and applications

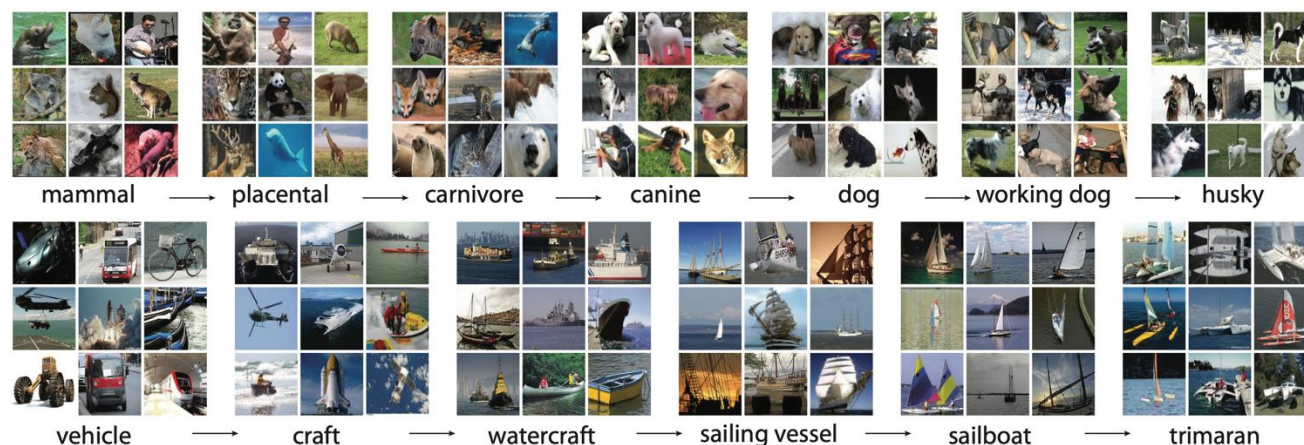
- Labeled data and augmentation
- Modern CNN models
- Object detection
- Semantic Segmentation

Labeled data and augmentation

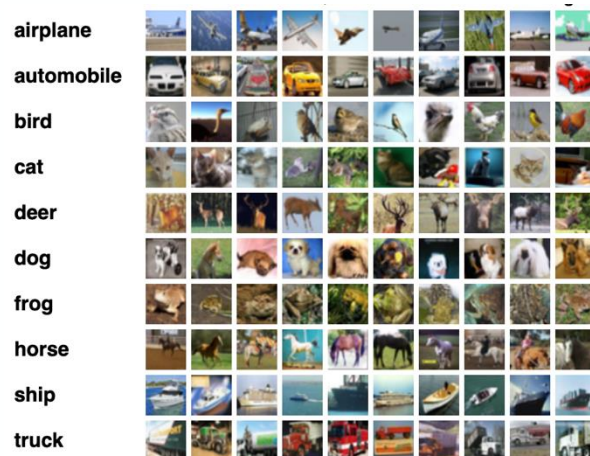


Leaps in training data

IMAGENET



CIFAR-10



State of the art in computer vision driven by large training data

- ImageNet (Li Fei-Fei) has been instrumental in advancing computer vision
- First released 2009 with 1 million examples and 1000 classes
- Labeled by workers on Amazon Mechanical Turk
- Relatively high resolution of 224×224 pixels
- Compare: CIFAR-100 only 60,000 images and 100 classes
- Now, LAION-5B has billions of images with additional metadata

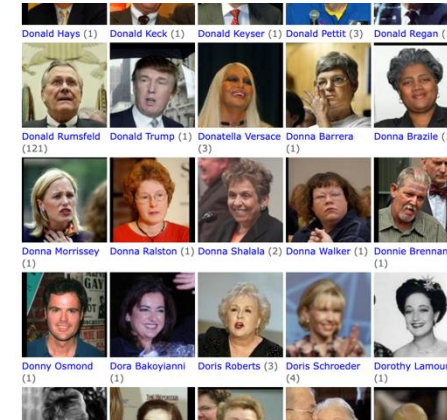
Other common datasets



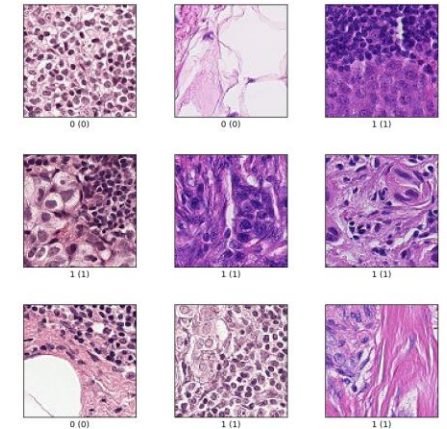
MNIST (National Institute of Standards and Technology)
 $n = 70,000, K = 10$



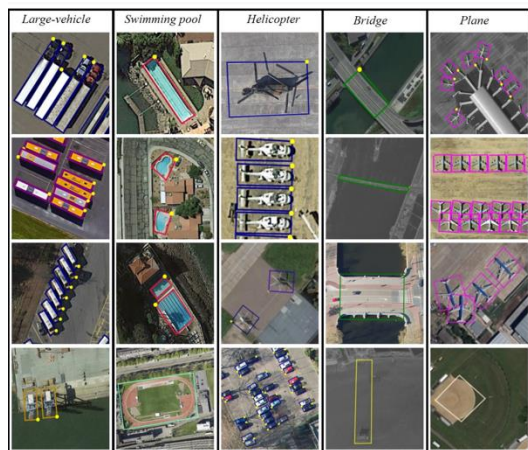
Fashion-MNIST (Zalando)
 $n = 70,000, K = 10$



Labeled Faces in the Wild
 Home (U Mass Amherst)
 $K = 5749$ (people)
 $n = 13,233$ (images)



patch_camelyon (Veeling et al.)
 $n = 327,680$
 $K = 2$ (metastatic tissue)



DOTA (Ding and Xia)
 11,268 images and
 1,793,658 instances
 $K = 18$

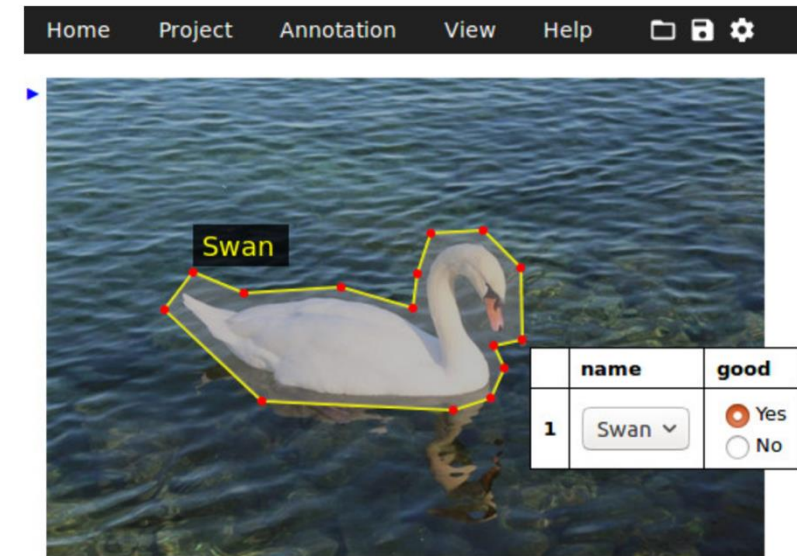
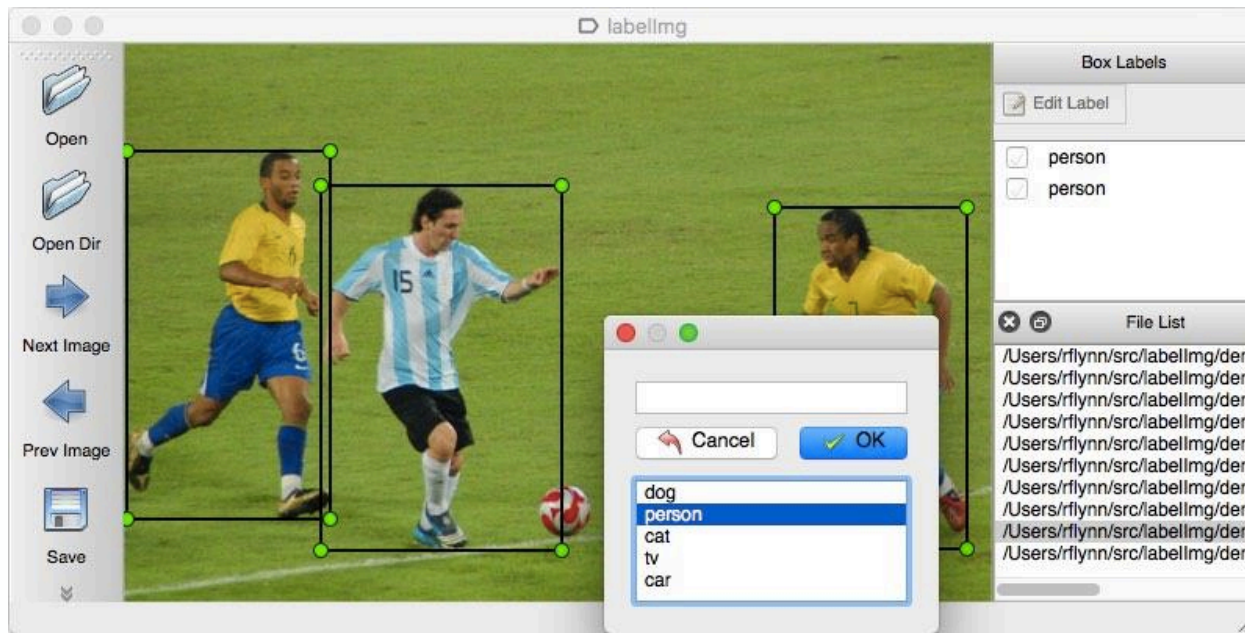


COCO [Common Objects in Context] (Microsoft)
 330K images (>200K labeled)
 1.5 million object instances
 80 object categories
 91 stuff categories
 5 captions per image
 250,000 people with keypoints

Data labeling

Annotating your own domain-specific dataset

- Many (open source and paid) tools available online, or write your own
- Tools use assistance to speed up labeling



Basic image Annotation

Data labeling

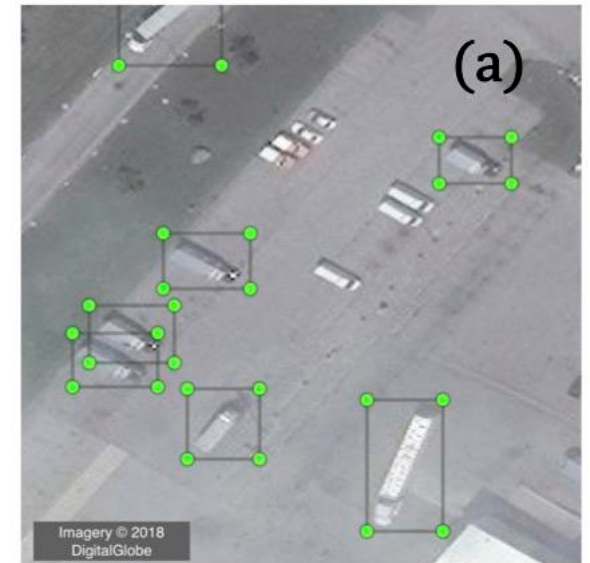
Considerations

- How much data is enough depends on the task
- Limitations: resources you have available
- Example truck detection in satellite images: >5000 examples manually labeled as a PhD student
- Depending on the project, you may need hundreds of hours of manual labor
- Who labels:
 - Project team (researchers)
 - Trained research assistants
 - Crowd-sourcing with Amazon Mechanical Turk etc.
- Huge differences in quality depending on the task
- **Data quality is important!**
- Some tasks can be translated to work with crowd sourcing

Data labeling

Quality control

- Spend sufficient time and thought to develop the annotation scheme/codebook
- Annotation scheme often the main scientific contribution in applied research
- Annotation scheme cannot be changed in the middle of the process
- Label examples yourself before and during developing the annotation scheme
- Carefully develop instructions and develop a rule-book
- Spend time training annotators; use small pilots
- Have parts of the data annotated several times
- Discuss edge cases with annotators
- Monitor quality with inter-annotator agreement measures
- Publish your data (if allowed)

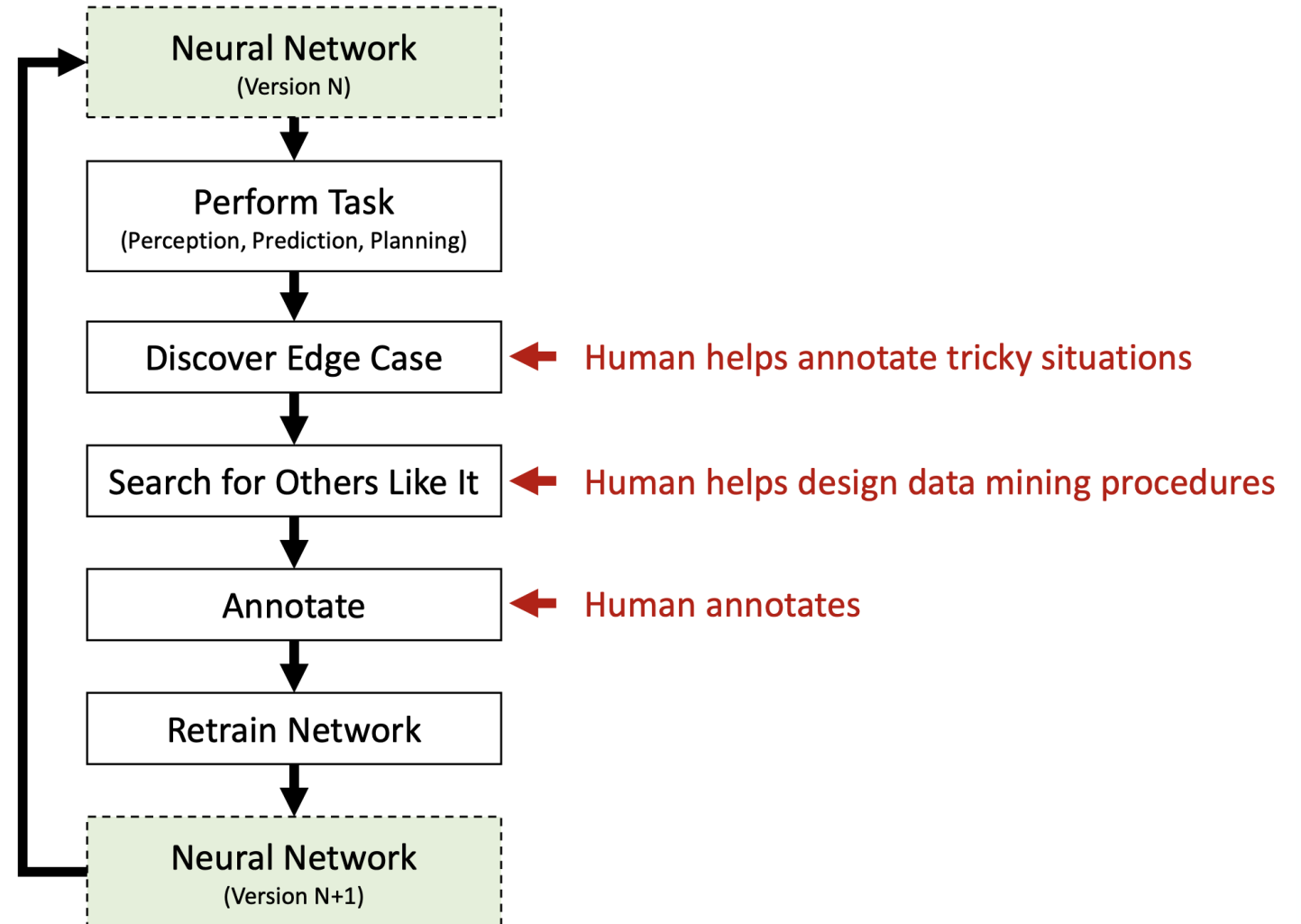


Example: What is a truck, what is a van?

Labeling with active learning

Active learning

- In active learning, annotation continues while the model is being trained
- Learning algorithms query the user (“teacher”) for labels
- Edge cases are labeled to provide more data where the model fails
- Can reduce the total amount of training data needed
- Risk of oversampling uninformative examples



Model-assisted labeling



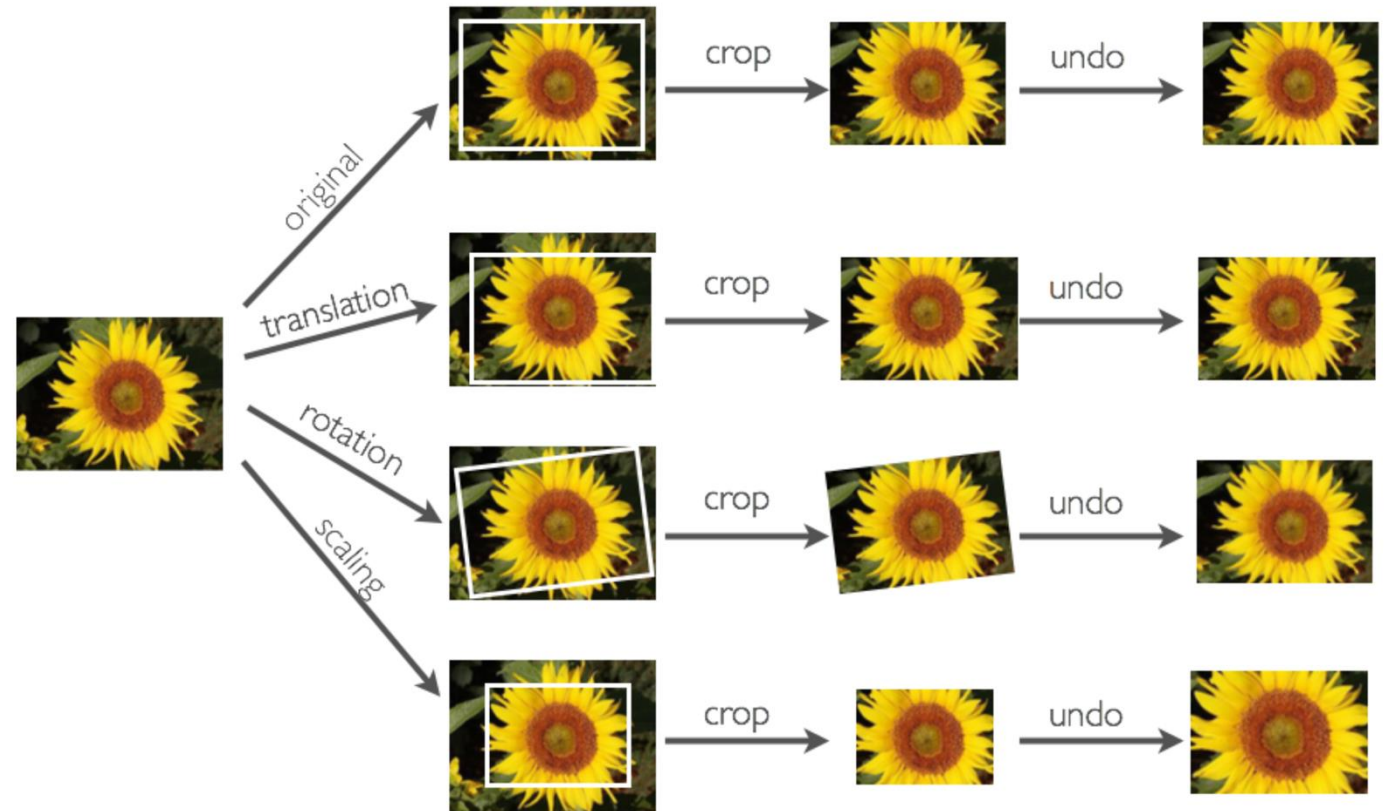
Fast segmentation

- Models are integrated into the annotation process to make annotation faster
- Segmentation example: instead of drawing a polygon, one simply clicks into an object
- Manual correction after

Data augmentation

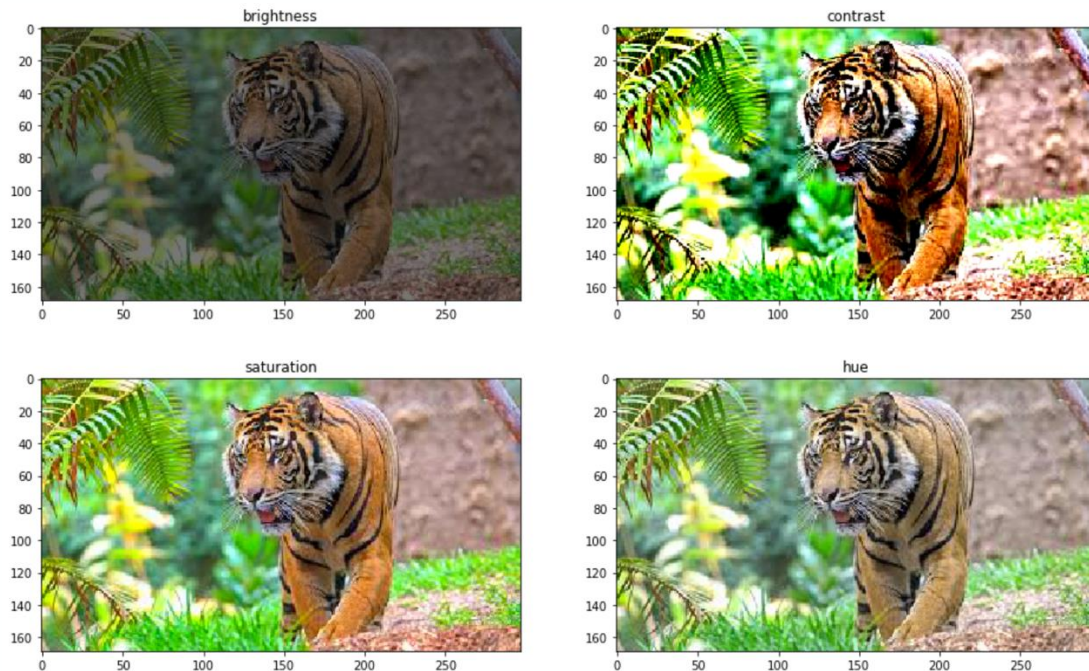
Generating additional examples

- Generating random images based on existing training data to improve the generalization ability of models
- Only apply image augmentation to training examples
- Do not use image augmentation with random operations during prediction
- DL frameworks provide many different image augmentation methods, which can be applied simultaneously

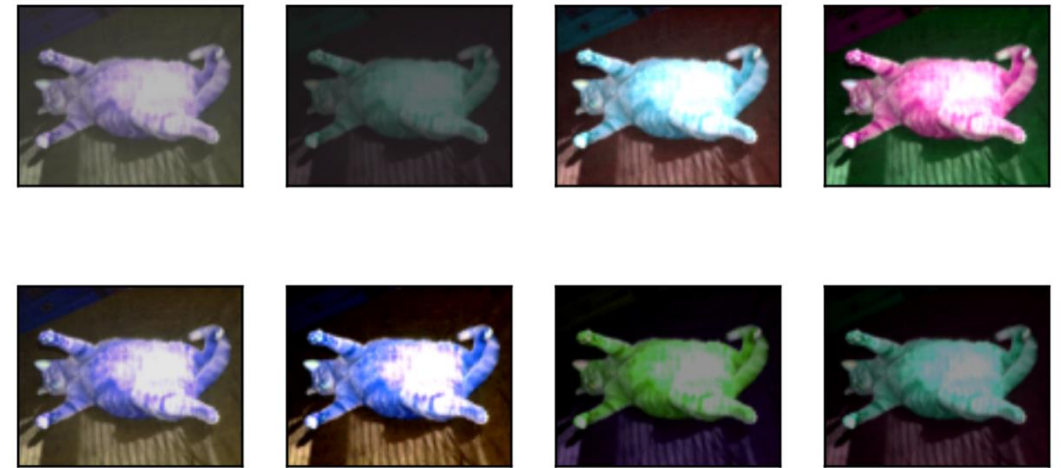
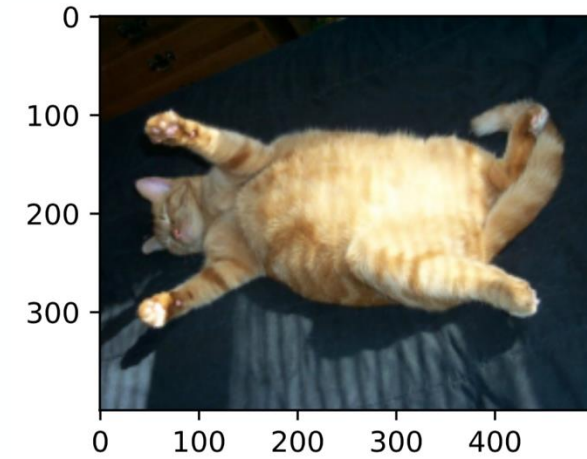


Data augmentation

Generating additional examples: Changing colors



Color augmentations on image of a tiger

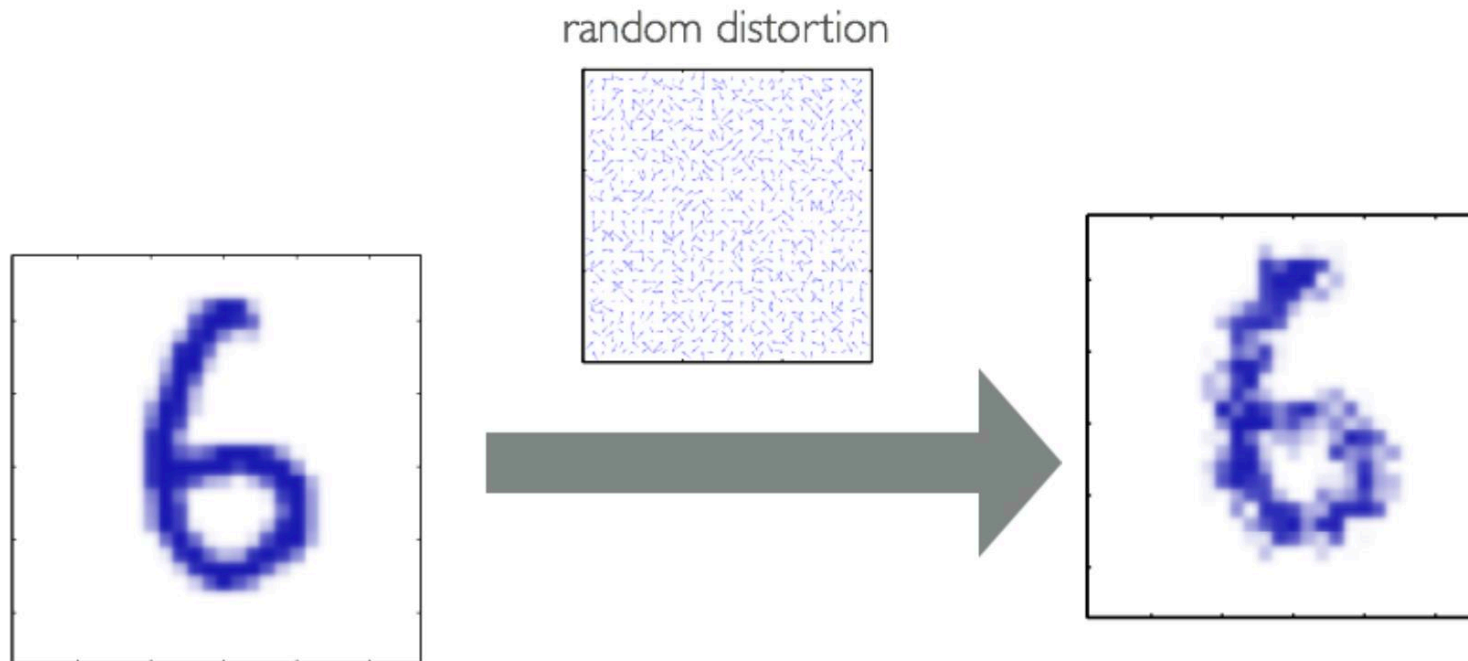


Data augmentation

Elastic distortions

Can add “elastic” deformations (useful in character recognition)

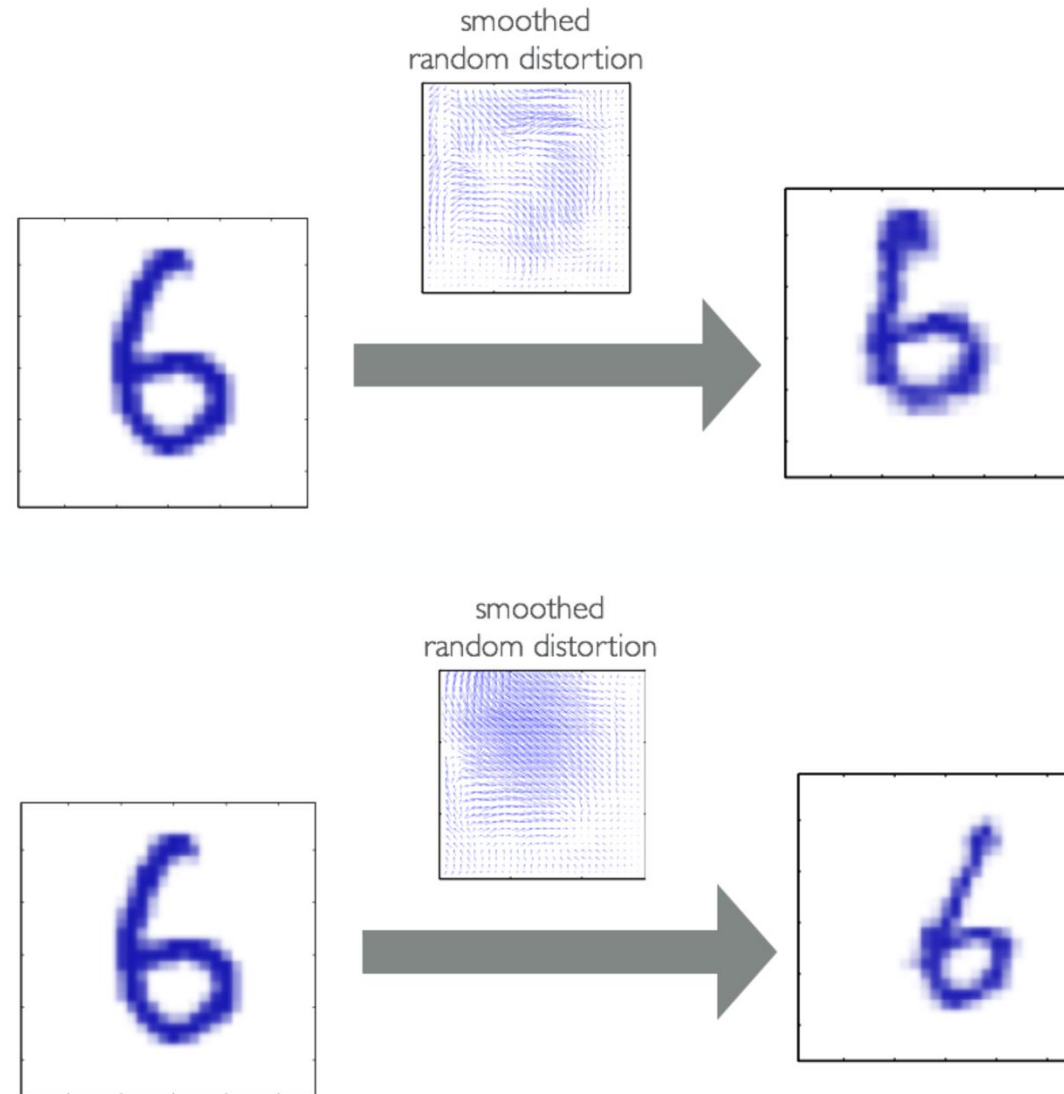
We can do this by applying a “distortion field” to the image that specifies where to displace each pixel value



Data augmentation

Elastic distortions

Smoothed random distortions

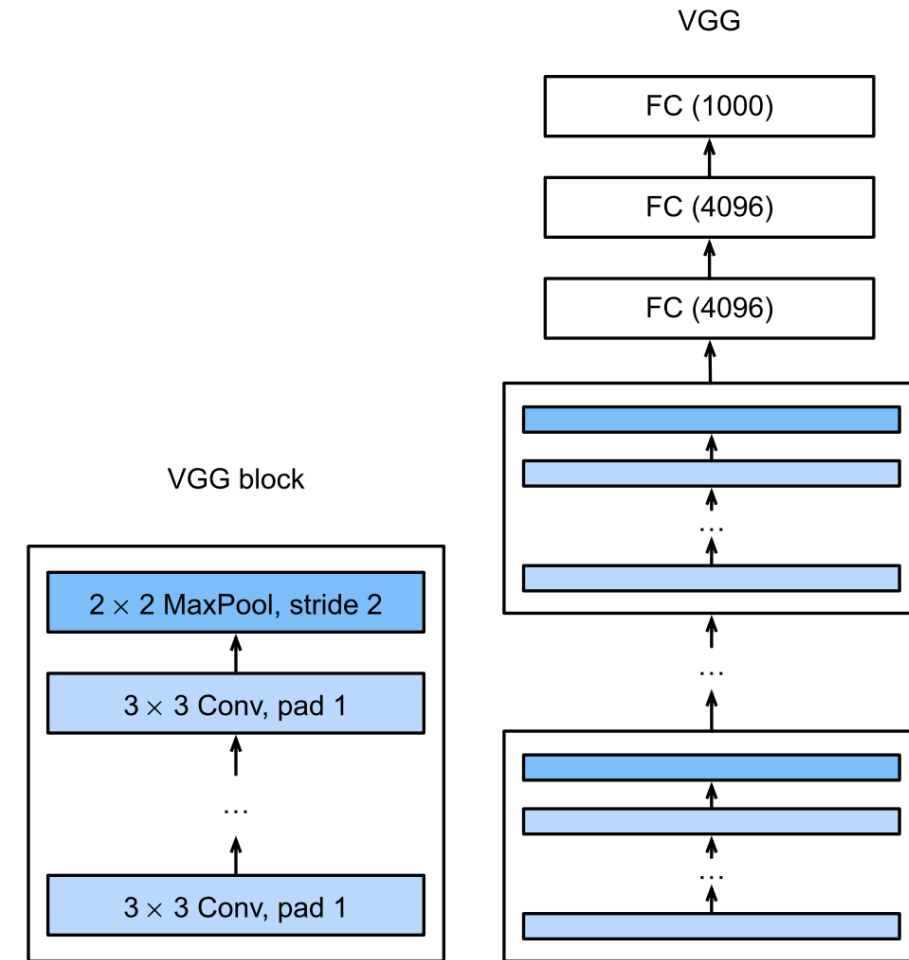


Modern CNN models

VGG

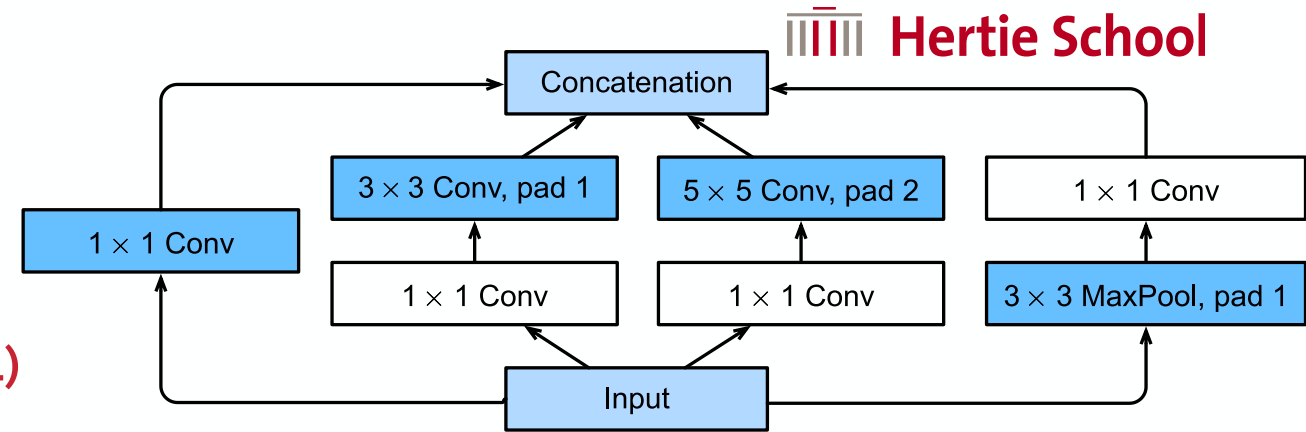
Introducing blocks and deep networks (2014)

- Basic CNN block: (i) a convolutional layer with padding to maintain the resolution, (ii) a nonlinearity such as a ReLU, (iii) a pooling layer such as max-pooling to reduce the resolution
- Problem: resolution reduces very fast with many pooling layers
- **VGG block:** VGG block consists of a sequence of convolutions with 3×3 kernels with padding of 1 (keeping height and width) followed by a 2×2 max-pooling layer with stride of 2 (halving height and width after each block)
- Thinking shifted from layers to blocks of layers
- Preference for deep and narrow networks (small convolutions)
- VGG-11: 8 convolutional layers and 3 fully connected layers



GoogLeNet

Multi-branch networks: Inception blocks (2014)

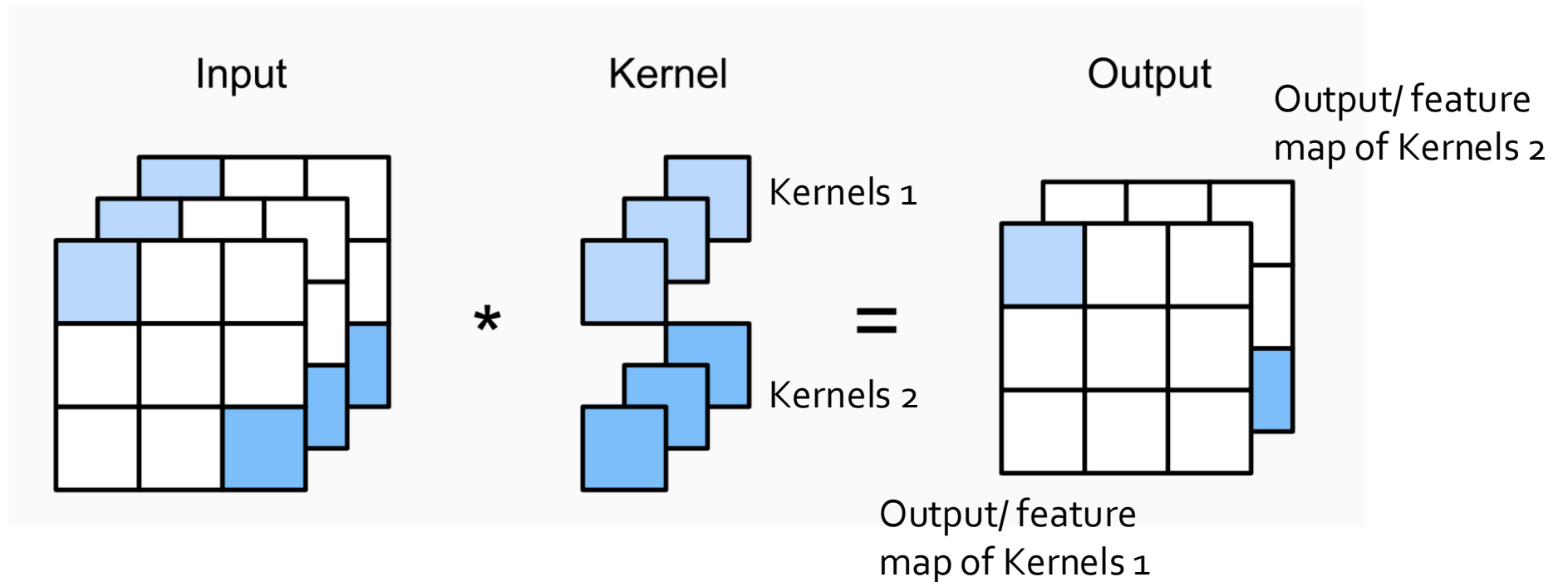


- The first three branches use convolutional layers with window sizes of 1×1 , 3×3 , and 5×5 to extract information from different spatial sizes.
- The fourth branch uses a 3×3 max-pooling layer
- The 1×1 convolution of the input or output reduce the number of channels, reducing the model's complexity.

GoogLeNet

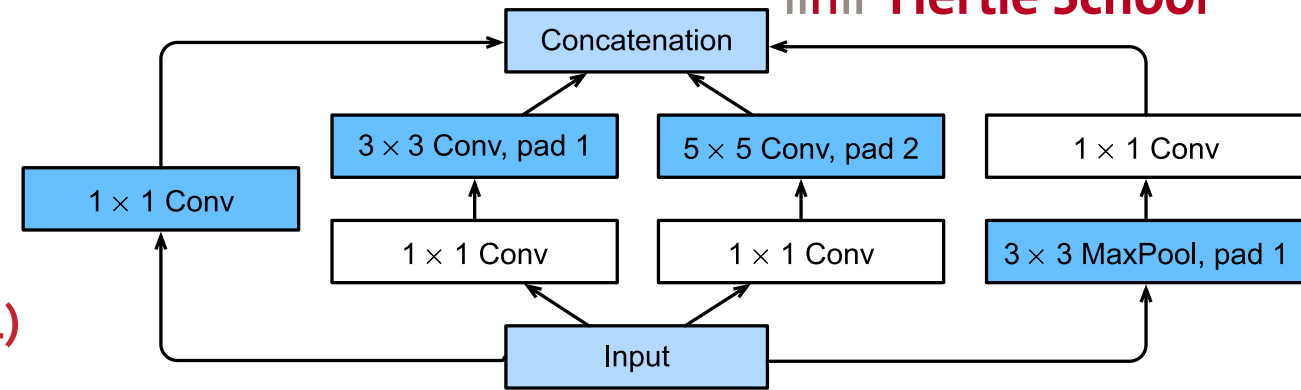
1x1 convolution

- Only purpose is to change the number of channels
- Followed by a non-linear activation function



GoogLeNet

Multi-branch networks: Inception blocks (2014)

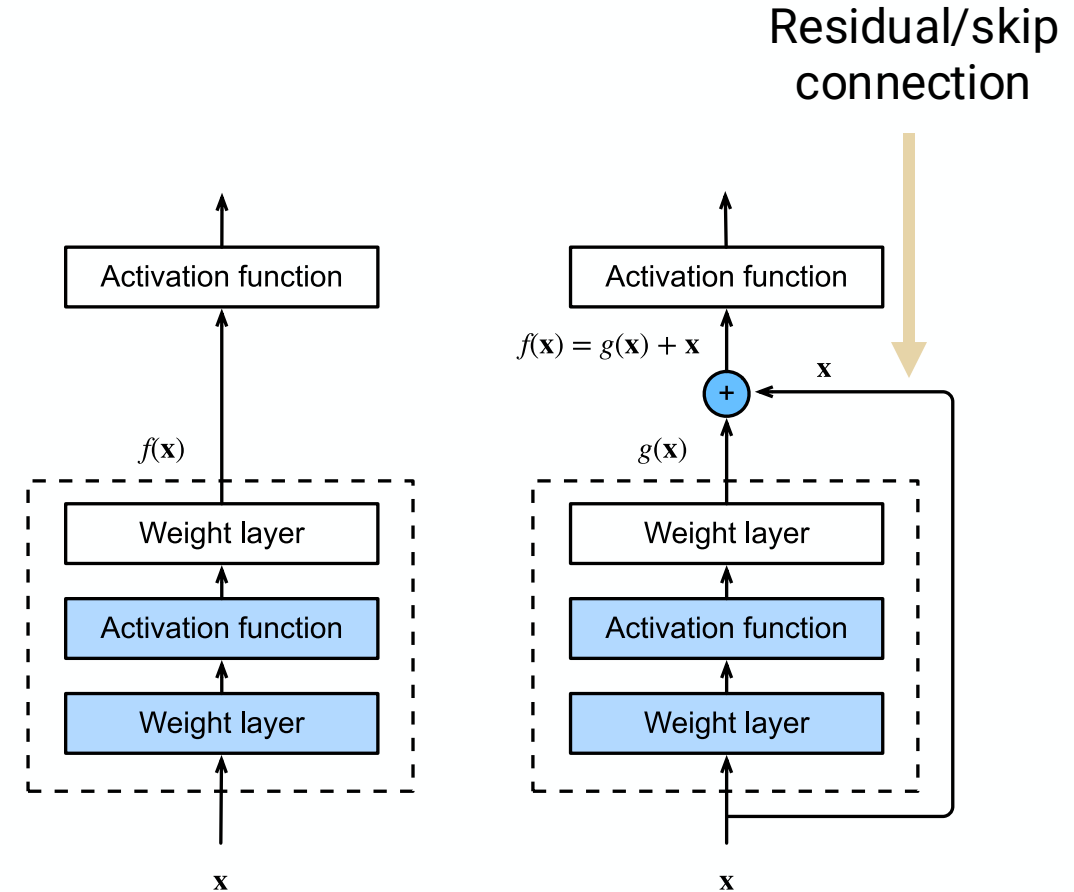


- The first three branches use convolutional layers with window sizes of 1×1 , 3×3 , and 5×5 to extract information from different spatial sizes.
- The fourth branch uses a 3×3 max-pooling layer
- The 1×1 convolution of the input or output reduce the number of channels, reducing the model's complexity.
- The four branches all use padding to give the input and output the same height and width
- The outputs along each branch are concatenated along the channel dimension (stacked outputs of all four resulting in a larger number of channels)
- Hyperparameters of the Inception block: number of output channels per branch. Will allocate capacity among convolutions of different size.
- Computationally complex, but cheaper to compute than its predecessors while simultaneously providing improved accuracy.

Residual networks (ResNet)

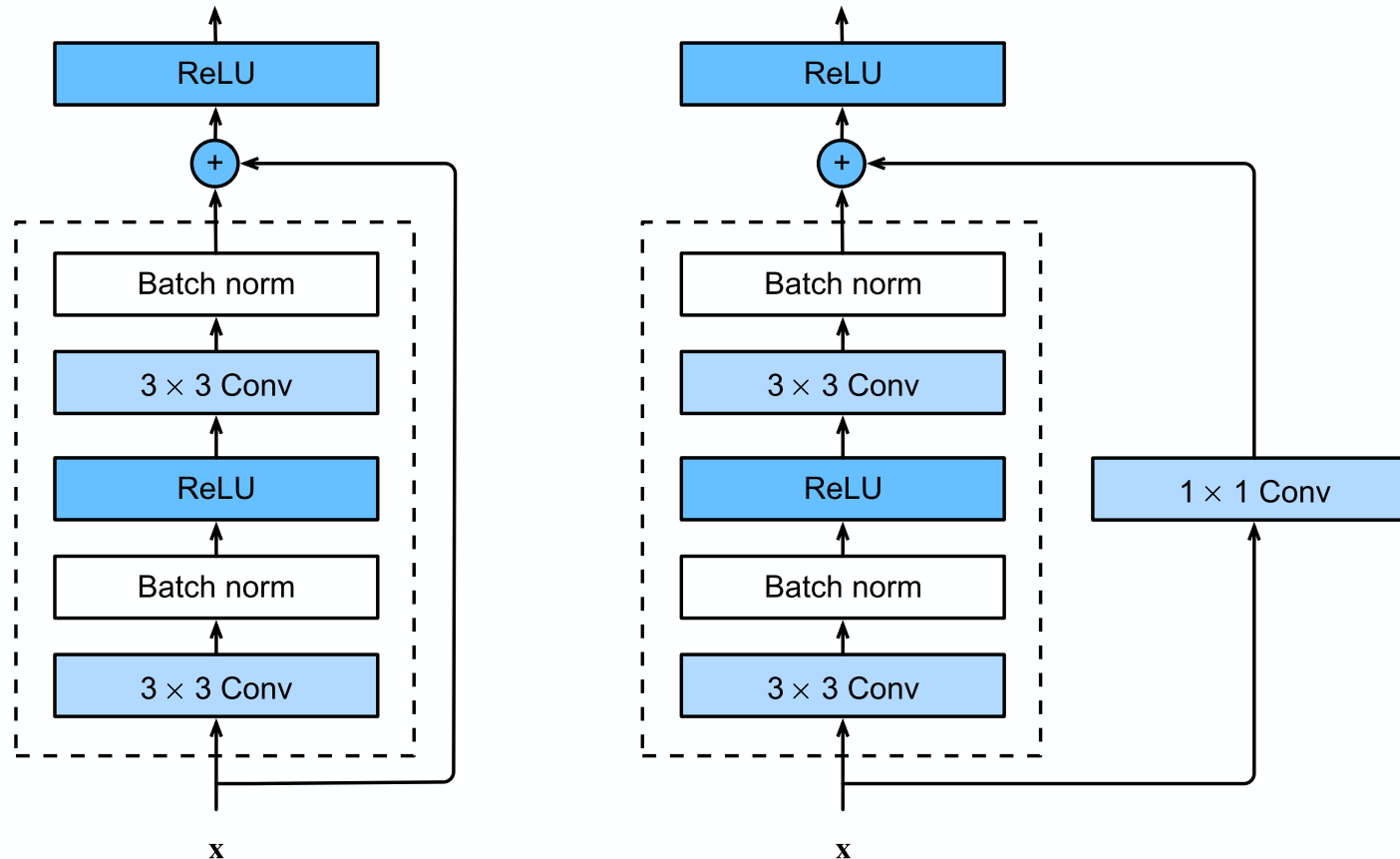
Adding residual/skip connections (2016)

- Idea: We want a larger network to be at least as good as the smaller one. Adding the identity function to the new layer will make it at least as effective as the original model
- Introducing the **residual block**
- Image: portion of the block in dotted lines learns the **residual mapping** $g(x) = f(x) - x$ in residual block
- If $f(x)$ is close to x the residual is much easier to learn
- Special two-branch case of inception block
- Improved also recurrent networks (next session), transformers (NLP sessions), and graph neural networks



Regular block (left) and residual block (right)

Residual networks (ResNet)



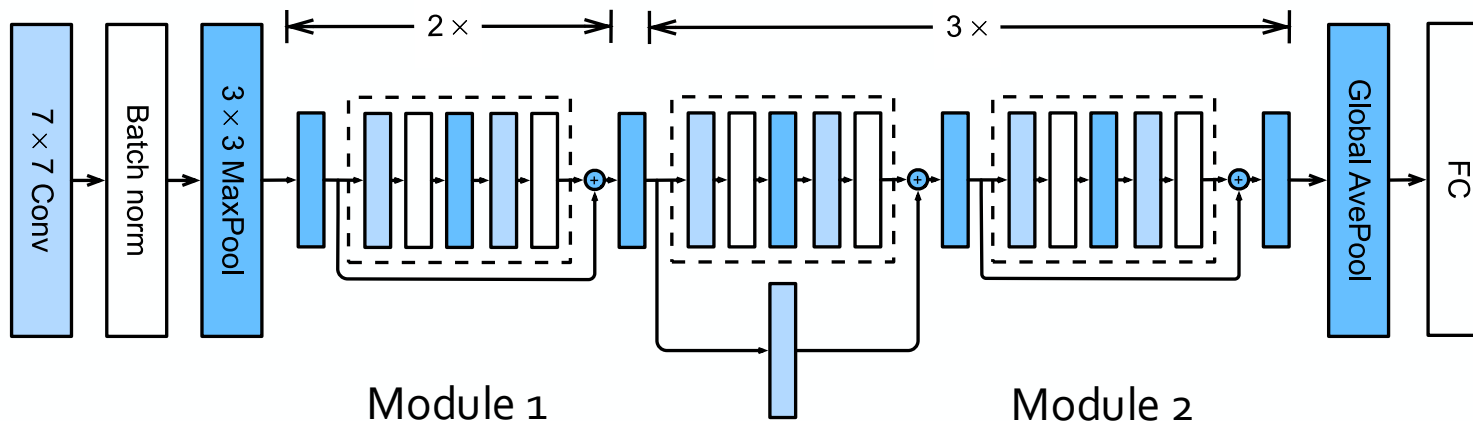
ResNet block

- 3×3 convolutions inspired by VGG
- By adding a 1×1 convolution in the residual connection, we can change the number of channels
- Needed for when the 3×3 convolutions change number of channels

Residual networks (ResNet)

ResNet model

- ResNet-18 has $2 \times$ Module 1 (1 block) and $3 \times$ Module 2 (2 blocks) in addition to initial 7×7 convolutional layer and final fully-connected layer
- Each residual block has 2 convolutional layers
- Total of $2 \cdot 2 + 3 \cdot (2 + 2) + 1 + 1 = 18$ layers with weights



- **Global average pooling** layer takes the average of each feature map (channel) as a feature
- Sometimes that is fed directly into the softmax
- Need K feature maps for K classes
- Prevents overfitting

Residual networks (ResNet)

Summary

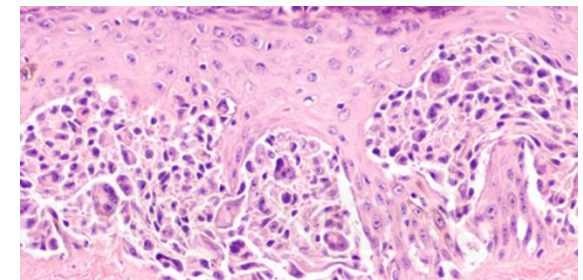
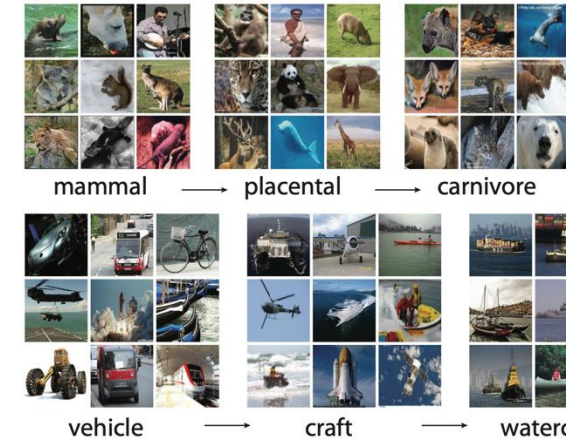
- Model family with e.g., 152-layer ResNet-152
- Residual connections allow to train much deeper networks
- Layers can be added during training because residual connections let the data pass through unchanged
- Very common as pre-trained models for fine tuning

Fine tuning pretrained models

Motivation

- Large models need energy, time, and money to be trained from scratch
- Basic image features can be quite generic
- We want to train on very large training datasets (e.g., ImageNet) to extract the most general image features
- Often not enough data in the target domain
- Apply *transfer learning* to transfer the knowledge learned from the *source dataset* to the *target dataset*.
- Example of digital pathology: Although most of the images in the ImageNet dataset (source) have nothing to do with rat tissue, a pre-trained model may also be effective for recognizing lesions in tissue (target dataset).
- Procedure: Take a pretrained model and fine tune it on target dataset.

Source dataset

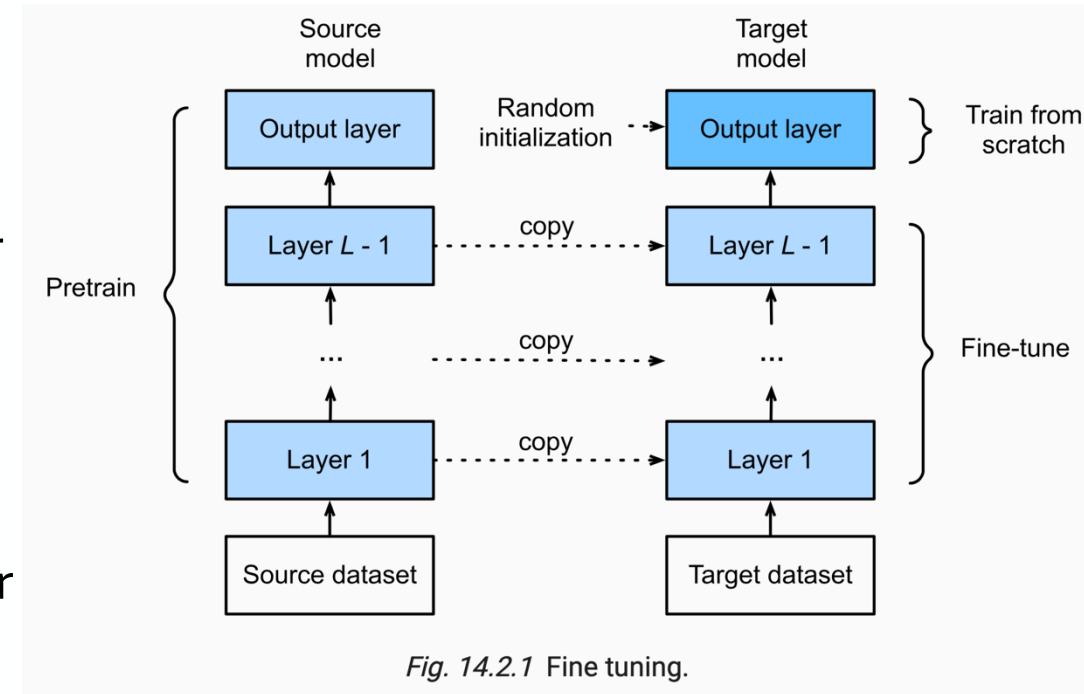


Target dataset

Fine tuning

General procedure

1. Pretrain a neural network model on a source dataset, or download pretrained model.
2. Create a new neural network model (target model) that copies all model designs and their parameters from the source model except the output layer.
3. Add an output layer to the target model, whose number of outputs is the number of categories in the target dataset. Then randomly initialize the model parameters of this layer.
4. Train the target model on the target dataset. The output layer will be trained from scratch, while the parameters of all the other layers are fine-tuned based on the parameters of the source model.



Often lower layers are also "frozen."

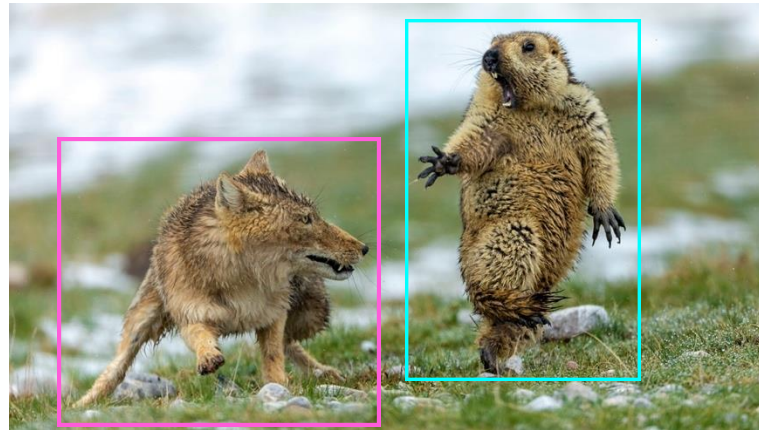
Computer vision tasks

Classification



Marmot

Object detection



Fox

Marmot

Instance segmentation



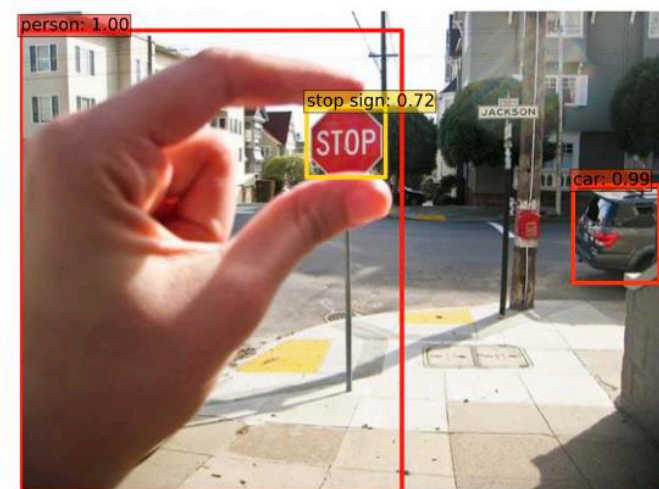
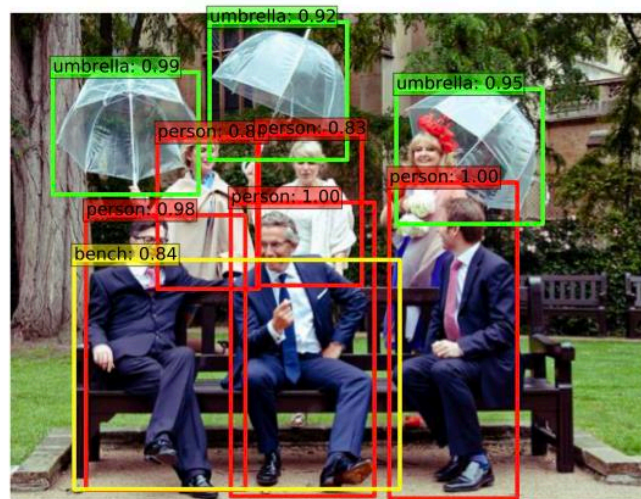
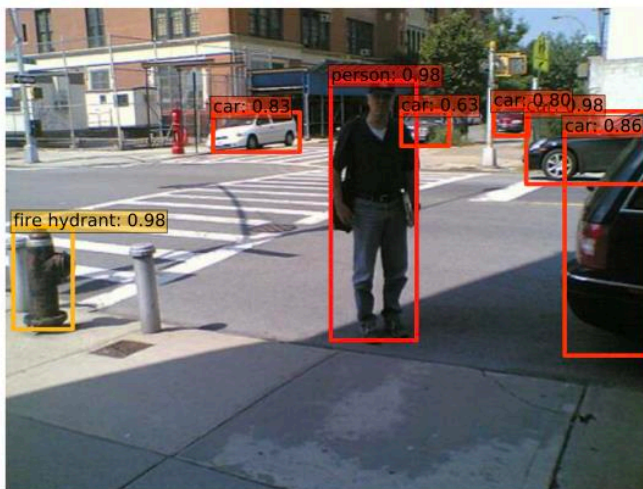
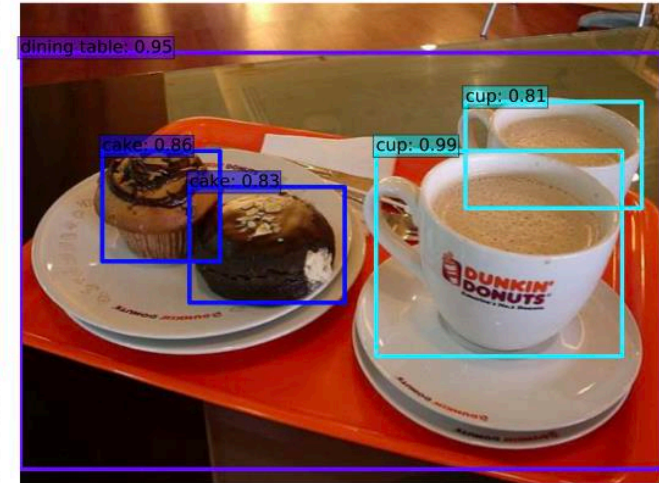
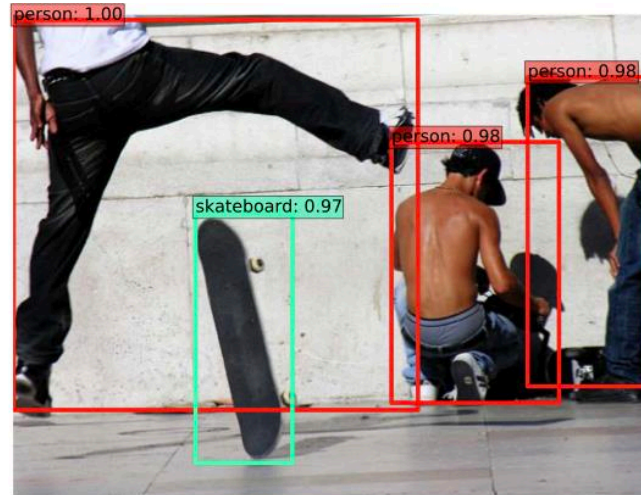
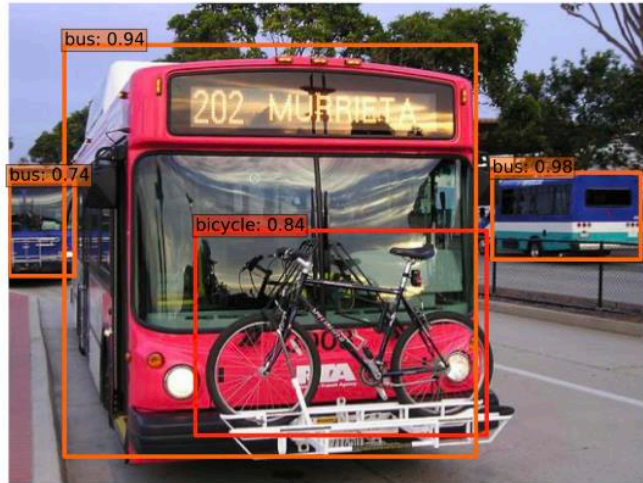
Fox

Marmot

Now we will talk about object detection and segmentation

Object detection

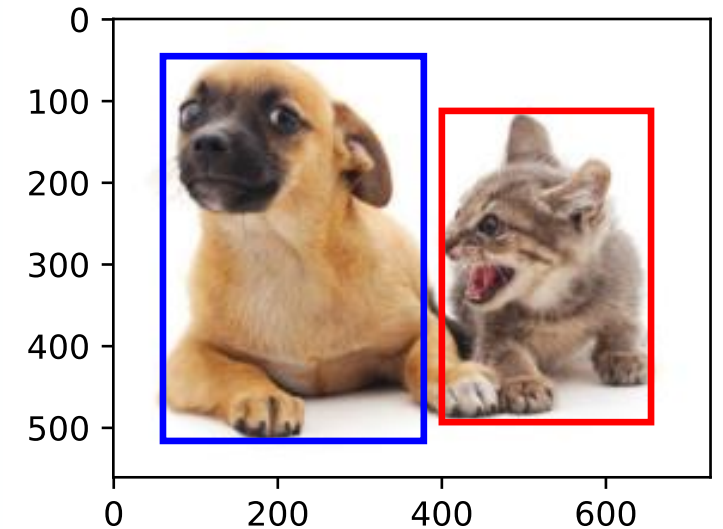
Object detection



Overview of object detection

Overview

- Object detection is also called “object recognition”
- Multiple objects in image
- Tasks:
 - Find the object(s) in the image
 - Draw a (tight) bounding box around the object
 - Determine the class of the object
- Applications:
 - Remote sensing of objects in satellite images such as planes
 - AVs identifying other vehicles, pedestrians, etc.
 - Face recognition in crowds
 - Target identification of robots
 - Counting, e.g. traffic counting, biodiversity monitoring, etc.

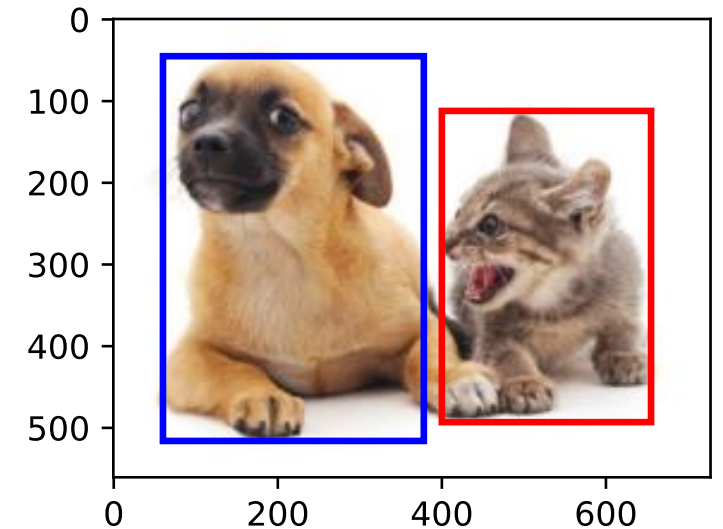


Overview of object detection

Rectangular bounding boxes

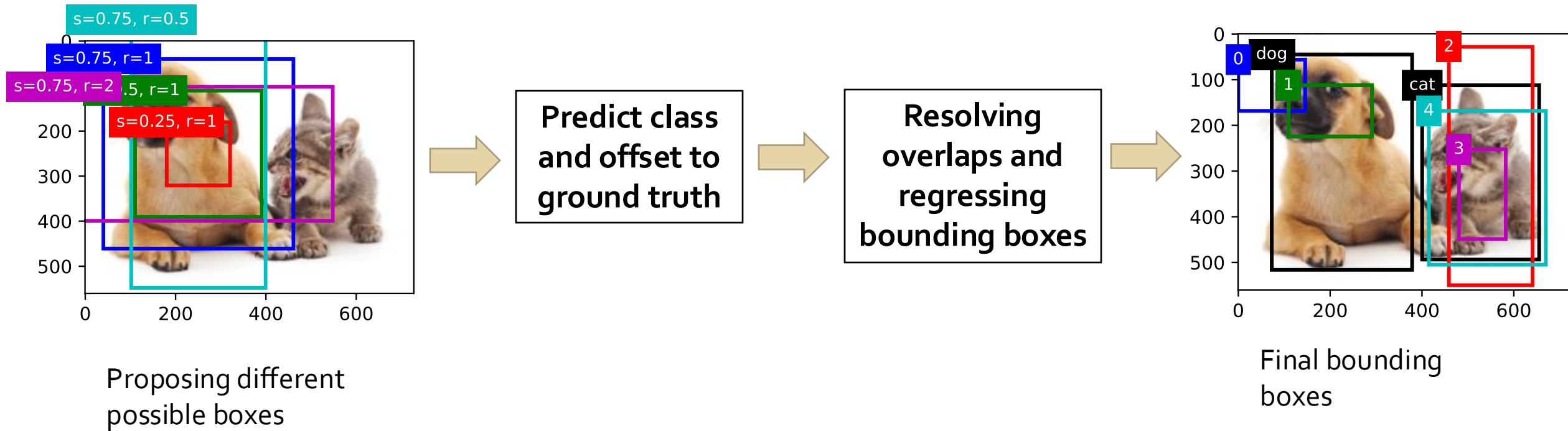
- Used for the spatial location of the object
- Two common and equivalent representations of the box
- Representation 1: The x and y coordinates of the upper-left corner of the rectangle and the such coordinates of the lower-right corner
- Representation 2: (x, y) -axis coordinates of the bounding box center, and the width and height of the box
- Coordinates are measured in pixels
- Boxes are learned with training data with ground truth bounding boxes

How do we find these boxes?



Overview of object detection

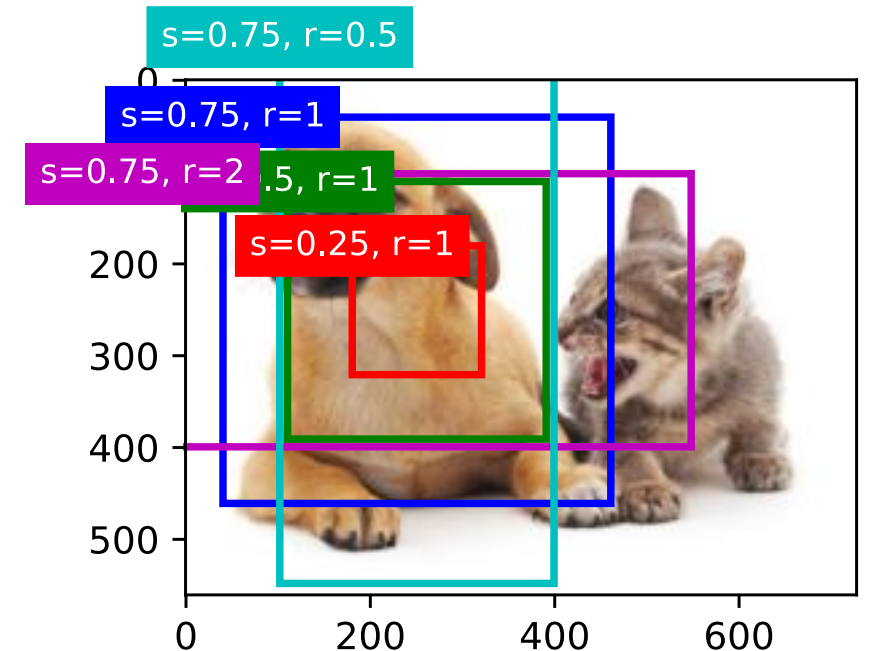
Basic workflow



Predicting boxes

Using anchor boxes

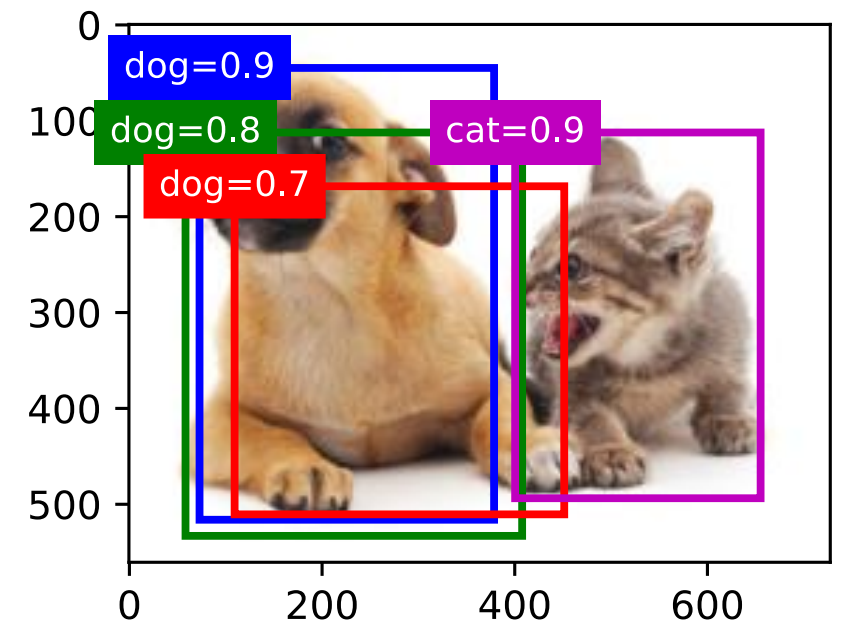
- Proposing a number of **anchor boxes** with different scales $s_1, \dots, s_n$ and aspect ratios $r_1, \dots, r_m$
- A small set of anchor boxes is chosen because otherwise it gets computationally intractable
- A set of different anchor boxes are evaluated at different points in the images
- During prediction, a class and an offset to a ground truth are predicted based on an anchor box
- A **predicted bounding box** is thus obtained according to an anchor box with its predicted offset



Predicting classes

Class prediction based on a predicted box

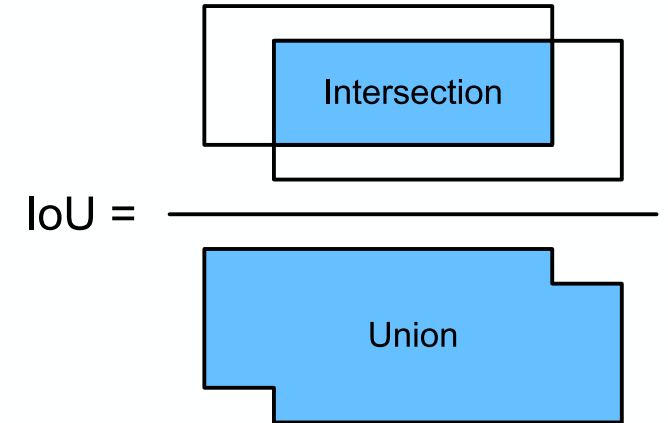
- Which class does the object shown in a predicted box belong to?
- Model outputs a score for each class (softmax output)
- Highest score is the predicted class
- The highest score is also called a **confidence score**
- Confidence scores are used to match predicted box to ground truth



Measuring agreement

Intersection over Union (IoU)

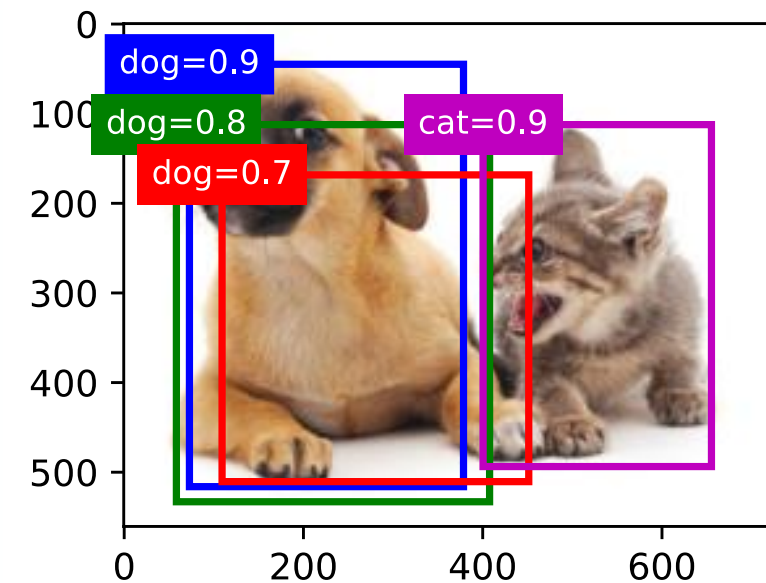
- Object detectors learn by comparison of predicted bounding boxes with ground truth bounding boxes
- How do we measure agreement?
- Jaccard index for two sets $\mathcal{A}$ and $\mathcal{B}$ is $J(\mathcal{A}, \mathcal{B}) = \frac{|\mathcal{A} \cap \mathcal{B}|}{|\mathcal{A} \cup \mathcal{B}|}$
- We can measure the similarity of the two bounding boxes by the Jaccard index of their pixel set (Intersection over Union (IoU))
- IoU is the ratio of the intersection area to the union area of two bounding boxes



Resolving overlaps

Non-maximum suppression

- Many similar (with overlap) predicted bounding boxes can be output for the same object, resulting from many anchor boxes
- Solution: merging similar predicted bounding boxes that belong to the same object by using **non-maximum suppression** (NMS):
 - Select the predicted bounding box with the highest confidence and remove all other predicted bounding boxes whose IoU with that first box exceeds a predefined threshold.
 - Select the predicted bounding box with the second highest confidence and remove all bounding boxes whose IoU exceeds the threshold
 - Repeat until all the predicted bounding boxes have been used.
 - Now, the IoU of any pair of predicted bounding boxes is below the threshold; no pair is too similar with each other.



SSD: Single Shot MultiBox Detector

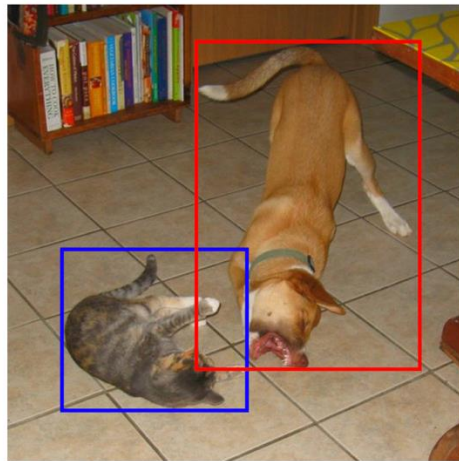
SSD by Liu et al (2015)

- A computationally efficient (fast) and high-performing object detector for multiple categories
- As accurate as slower techniques that perform explicit region proposals and pooling (including Faster R-CNN)
- Models relying on region proposals have an entire network to propose suitable bounding boxes for further classification, making them slow
- SSD uses anchor boxes to propose a number of boxes at low computational cost

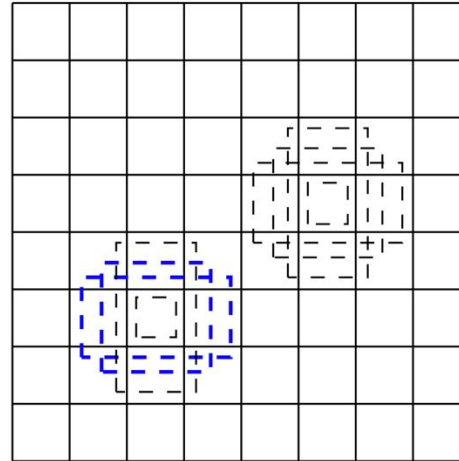
SSD: Single Shot MultiBox Detector

Anchor boxes

- Imagine the image gridded
- The same set of anchor boxes are evaluated at each center of the grid
- For each default box, we predict both the shape offsets and the scores (here called confidences) for all p object categories $((c_1, c_2, \dots, c_p))$.



(a) Image with GT boxes

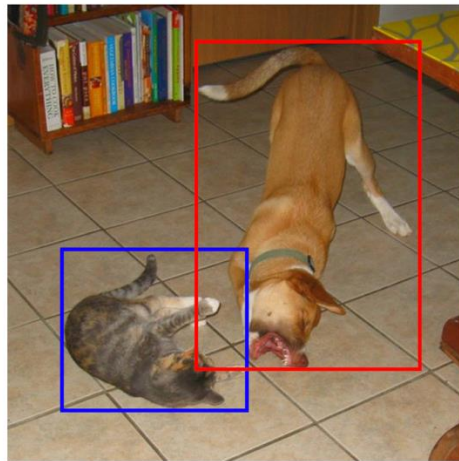


(b) 8×8 feature map

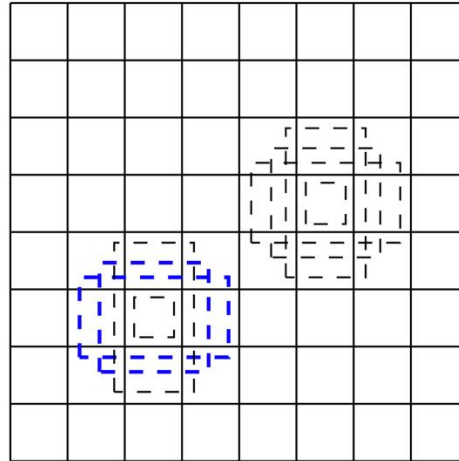
SSD: Single Shot MultiBox Detector

Multiscale anchor boxes

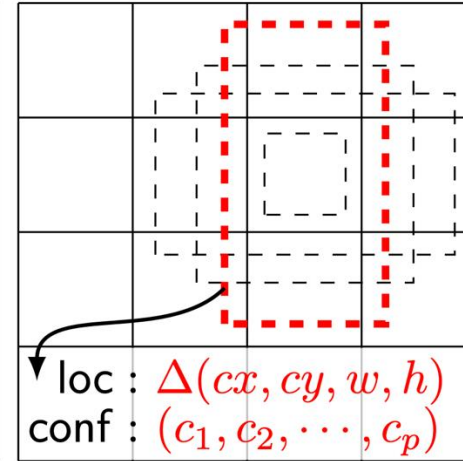
- We have different feature maps at different scales
- Anchor boxes are uniformly distributed over each feature map
- This allows to create bounding boxes at different scales



(a) Image with GT boxes



(b) 8×8 feature map



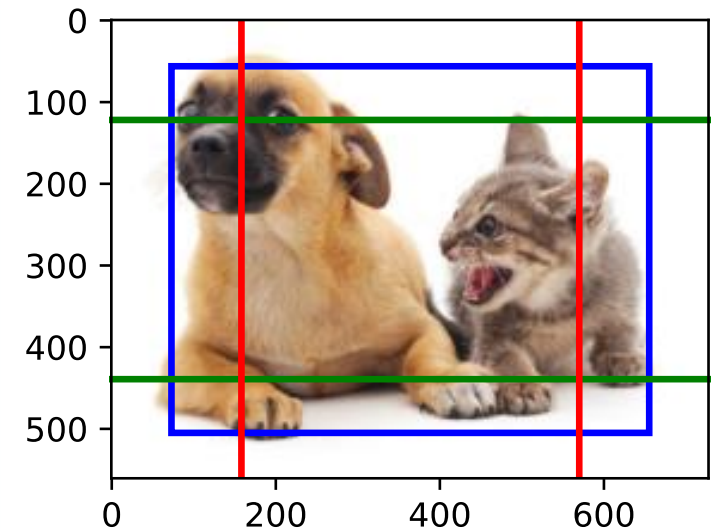
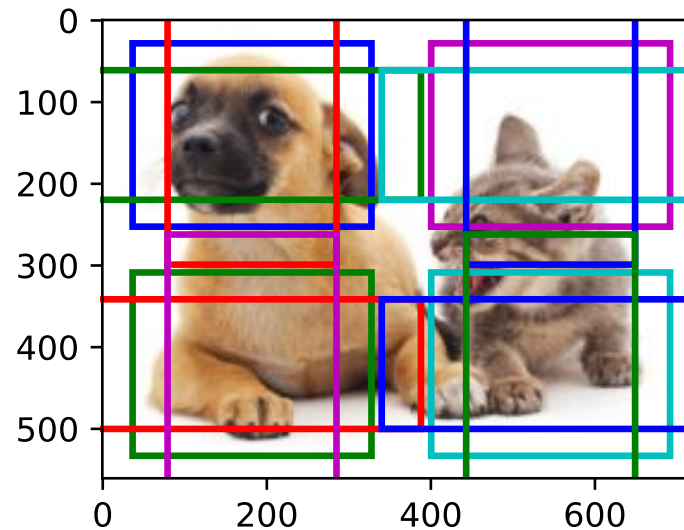
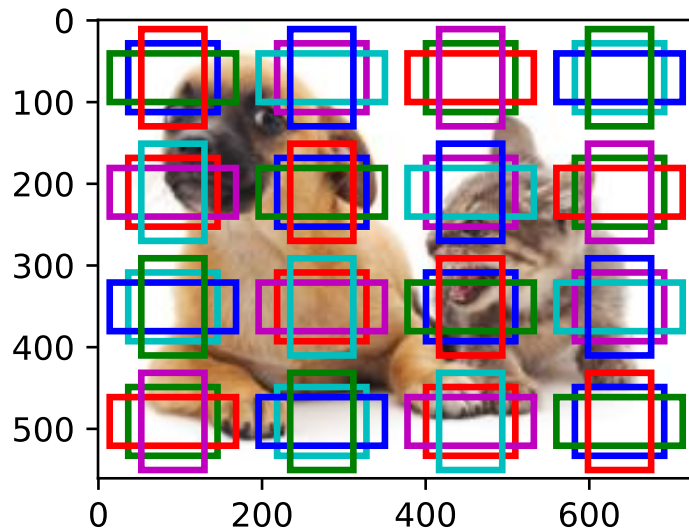
loc : $\Delta(cx, cy, w, h)$
conf : $(c_1, c_2, \dots, c_p)$

(c) 4×4 feature map

SSD: Single Shot MultiBox Detector

Multiscale anchor boxes

- We have different feature maps at different scales
- Anchor boxes are uniformly distributed over each feature map
- This allows to create bounding boxes at different scales

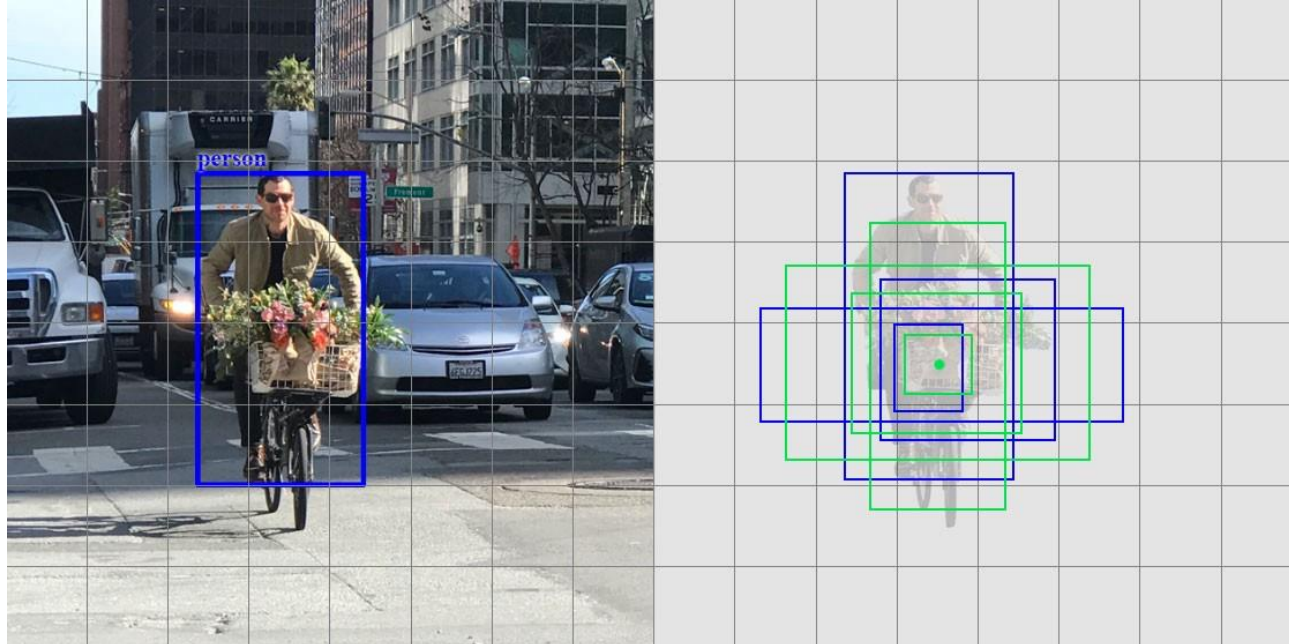


SSD: Single Shot MultiBox Detector

Prediction

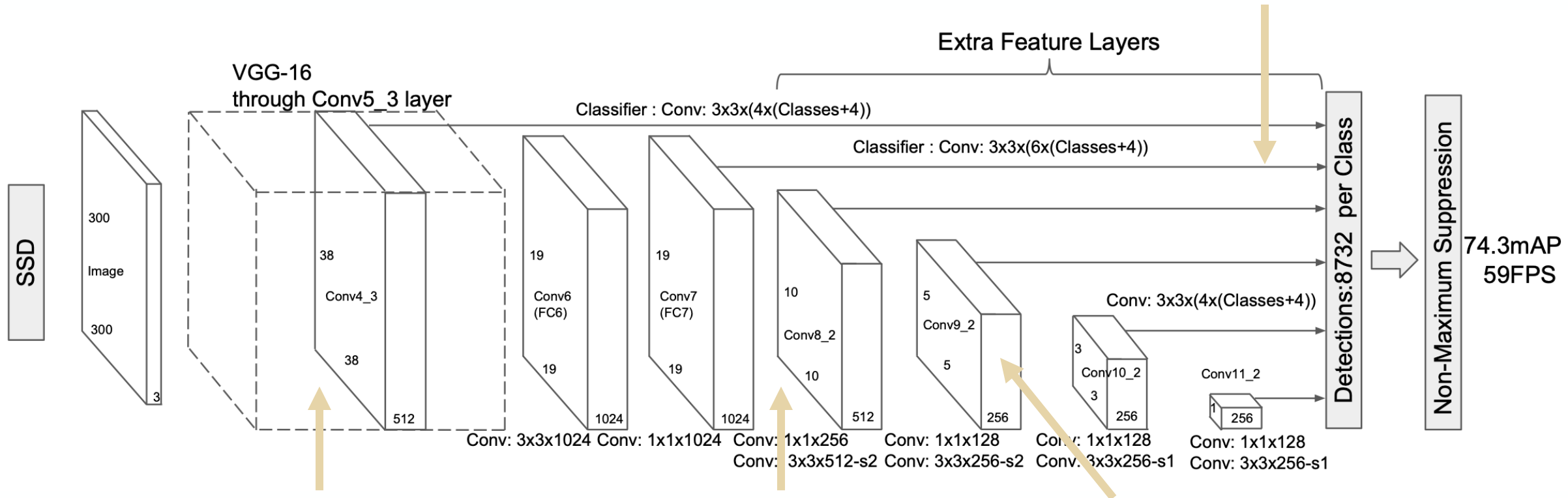
In one shot, the network predicts two things:

1. Offset of the true bounding box with anchor box
2. Confidence for each class



SSD: Single Shot MultiBox Detector

Each added feature layer can produce a fixed set of predictions (offset and confidence)



standard architecture used for high quality image classification (truncated before any classification layers)

convolutional feature layers decrease in size progressively and allow predictions of detections at multiple scales

At each feature map cell, we predict the offsets relative to the default box shapes in the cell, as well as the per-class scores

SSD: Single Shot MultiBox Detector

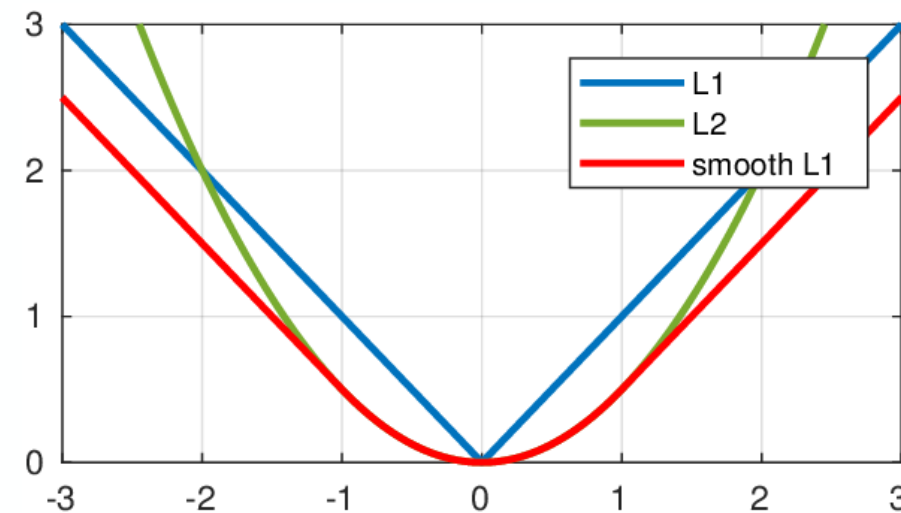
Training

- Matching algorithm: Any anchor box (with prediction) gets matched to ground truth with $IoU > 0.5$ & any ground truth gets matched to predicted box with highest IoU (irrespective of threshold)
- Loss function is weighted loss of the localization loss (loc) and confidence loss (conf)

$$L(x, c, l, g) = \frac{1}{N} (L_{conf}(x, c) + \alpha L_{loc}(x, l, g))$$

where N is the number of matched anchor boxes.

- L_{conf} is the softmax and cross-entropy loss over multiple confidences c
- L_{loc} is a Smooth L1 loss between the predicted box (l) and the ground truth box (g) parameters (a regression loss)



SSD: Single Shot MultiBox Detector

Training

Hard negative mining:

- After the matching step, most of the default boxes are negatives (match no ground truth, only background)
- This introduces a significant imbalance between the positive and negative training examples
- Instead of using all the negative examples, we sort them using the highest confidence loss for each default box and pick the top ones so that the ratio between the negatives and positives is at most 3:1.
- Leads to faster optimization and a more stable training

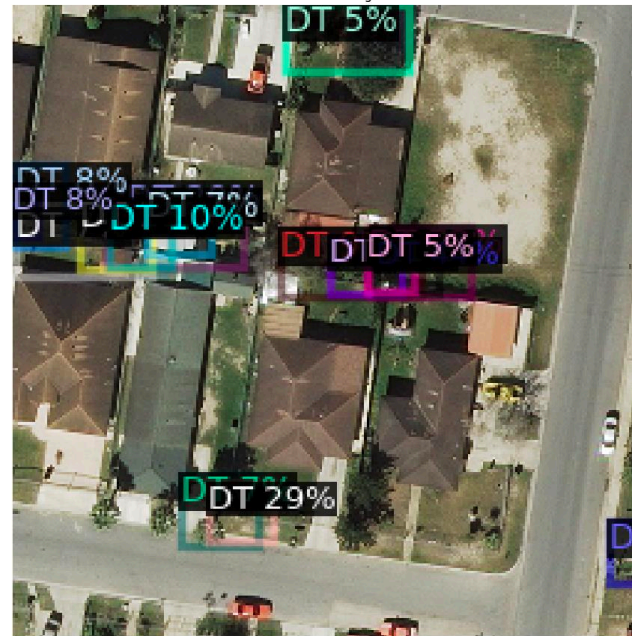
Data augmentation:

- In addition to original image, patches are included as well
- Some with a minimum Jaccard overlap with the set of objects to retain patches with objects
- Some random patches

Original Image



Predicted Image



What went wrong?

Original Image



Predicted Image



Semantic segmentation



Semantic segmentation

Overview

- Dividing an image into regions belonging to different semantic classes
- Labeling and prediction of semantic regions at the pixel level
- One of the most important semantic segmentation dataset is Pascal VOC2012

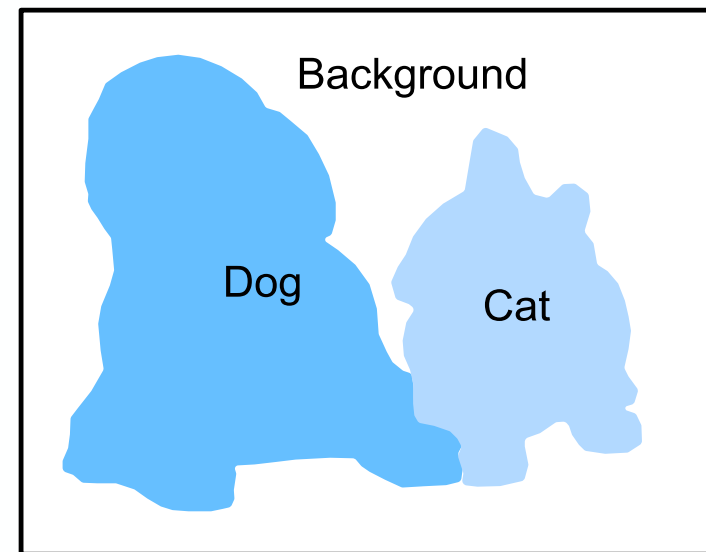


Image segmentation and instance segmentation

Image segmentation

- Divides an image into constituent regions
- Exploits correlation between pixels in the image
- Not necessarily supervised



Instance segmentation

- Simultaneous detection and segmentation
- Segmentation for each object separately



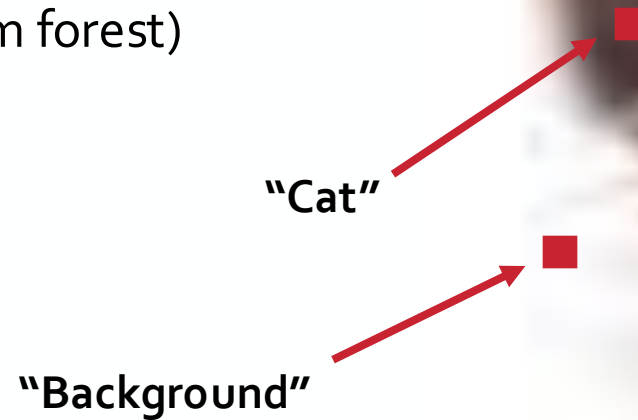
Semantic segmentation

Semantic segmentation

SEMANTIC segmentation
- image segmentation is a pixel-wide classifier

Pixel-wise classifier

- Every pixel in the image gets classified if it is belonging to the class “Cat”, “Dog”, or “Background”
- Traditional approaches used feature engineering such as pixel differences or features based on local filters with a classifier (e.g., random forest)

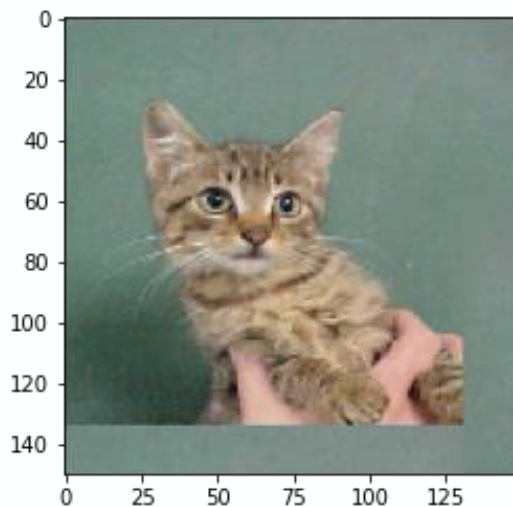


Semantic segmentation with deep learning

Idea

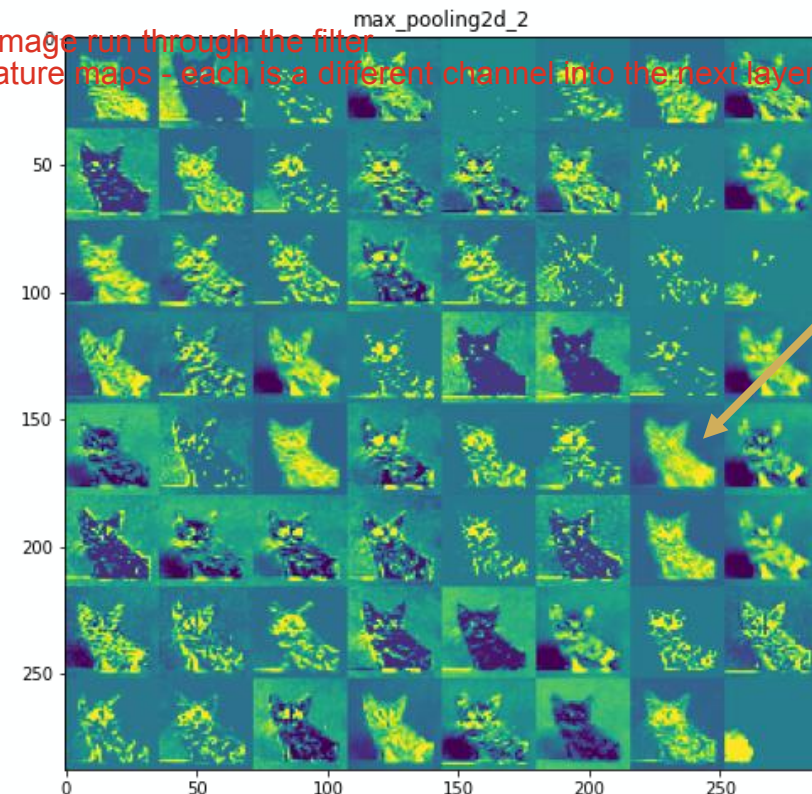
Feature maps of CNNs are very effective in extracting meaningful image features

- feature map = result of a convolutional layer: it's a high dimensional representation of the input image run through the filter
- in slide of cats - there are many cats because it's gone through many filters --> many parallel feature maps - each is a different channel into the next layer - each one is learning slightly different things



Original image

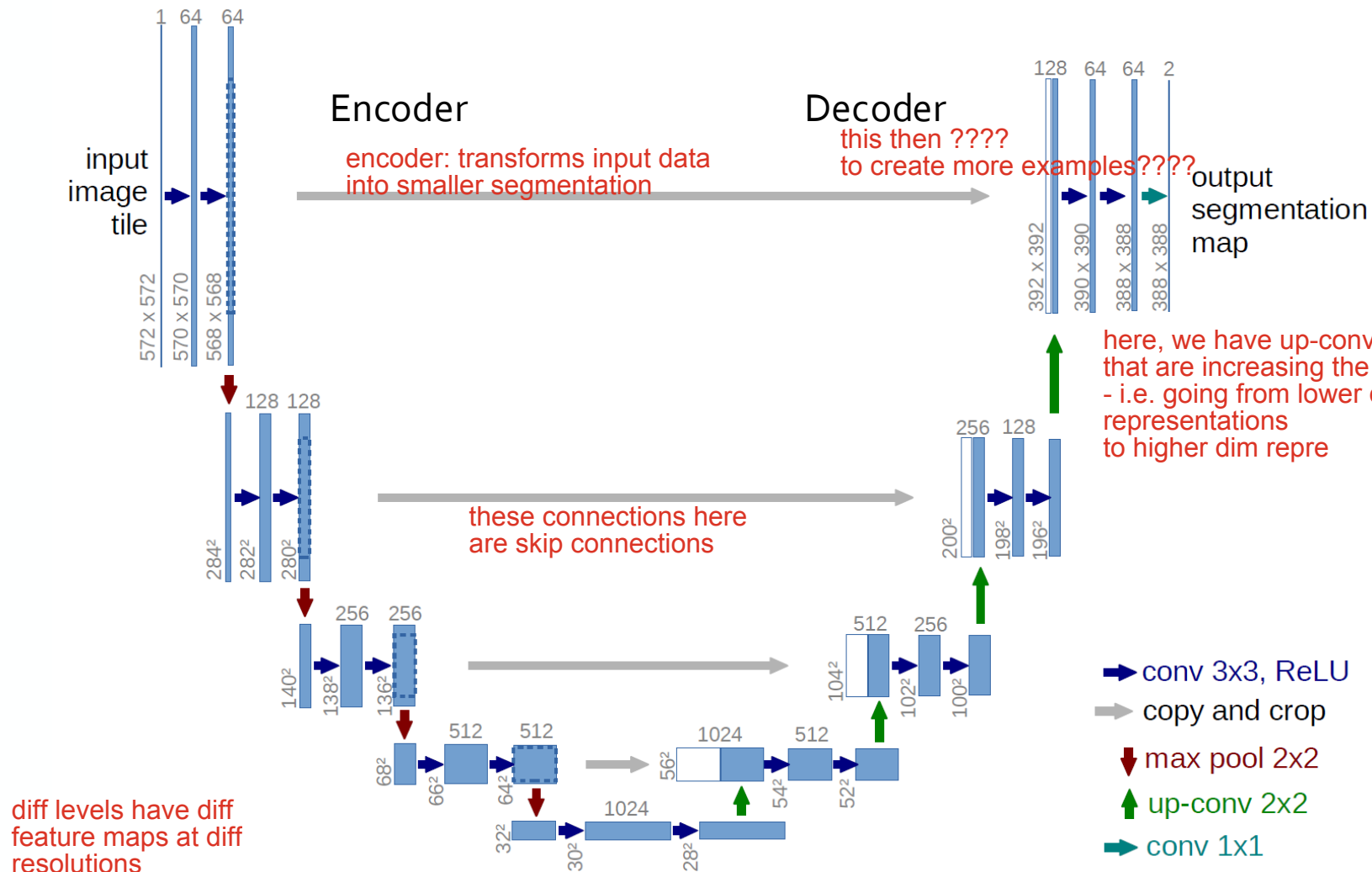
Two convolutional
and max pooling
layers



Looks like
segmentation
already

this is how semantic segmentation networks works: it's a smart trick
it exploits the feature maps

U-Net U-net = a famous network



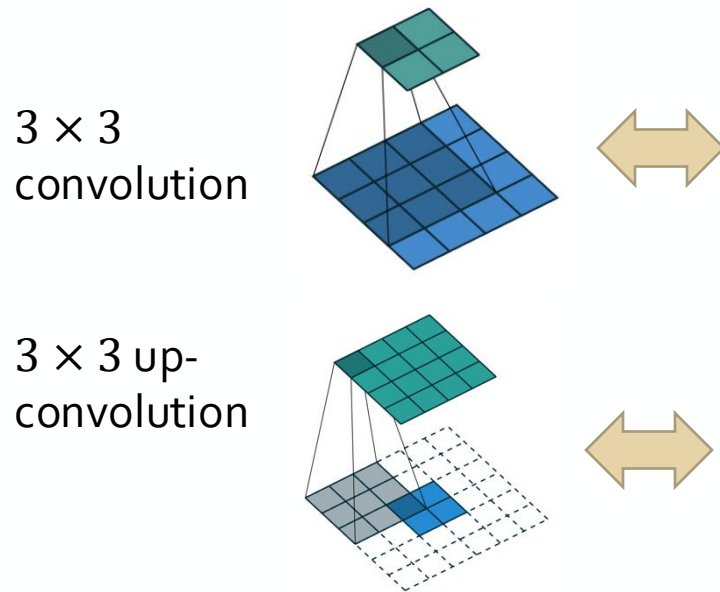
State-of the art semantic segmentation

- Originally developed for biomedical image segmentation
- Uses the feature activations of different feature maps
- Encoder-decoder design with skip connections
- Contains up-convolutions that increases the dimensionality (just like a convolution decreases the dimensionality)

U-Net

Up-convolution/ transposed convolution

- From something that has the shape of the output of some convolution to something that has the shape of its input while maintaining a connectivity pattern that is compatible with the convolution
- Needed for the decoding layer of the U-Net



Cross-correlation from illustration as a matrix multiplication

$$\begin{pmatrix}
 w_{0,0} & w_{0,1} & w_{0,2} & 0 & w_{1,0} & w_{1,1} & w_{1,2} & 0 & w_{2,0} & w_{2,1} & w_{2,2} & 0 & 0 & 0 & 0 & 0 \\
 0 & w_{0,0} & w_{0,1} & w_{0,2} & 0 & w_{1,0} & w_{1,1} & w_{1,2} & 0 & w_{2,0} & w_{2,1} & w_{2,2} & 0 & 0 & 0 & 0 \\
 0 & 0 & 0 & 0 & w_{0,0} & w_{0,1} & w_{0,2} & 0 & w_{1,0} & w_{1,1} & w_{1,2} & 0 & w_{2,0} & w_{2,1} & w_{2,2} & 0 \\
 0 & 0 & 0 & 0 & 0 & w_{0,0} & w_{0,1} & w_{0,2} & 0 & w_{1,0} & w_{1,1} & w_{1,2} & 0 & w_{2,0} & w_{2,1} & w_{2,2}
 \end{pmatrix}$$

the convolution here is being multiplied by the input flattened

...from this we can deduce the convolution

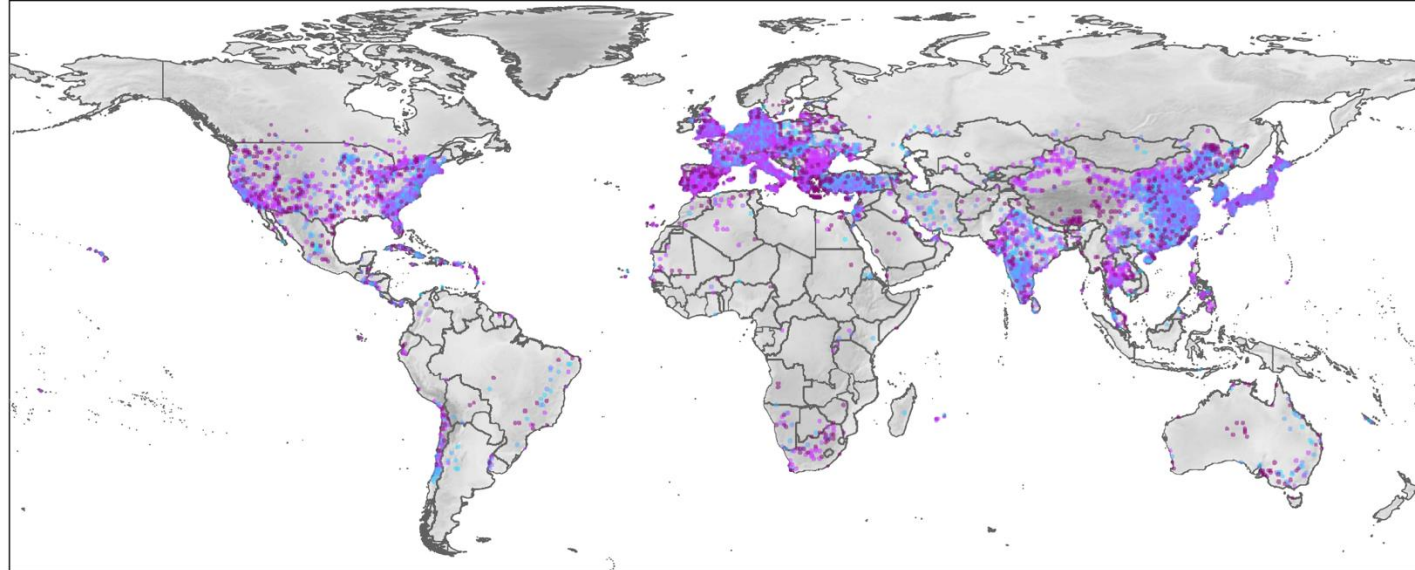
- we can transform these complex things into more simple matrix multiplication

multiplied with 16-dim flattened input

up convolution is the transpose of this matrix

- Transpose of the above matrix
- Equivalent to a cross-correlation with the 2 × 2 input padded with a 2 × 2 border of zeros

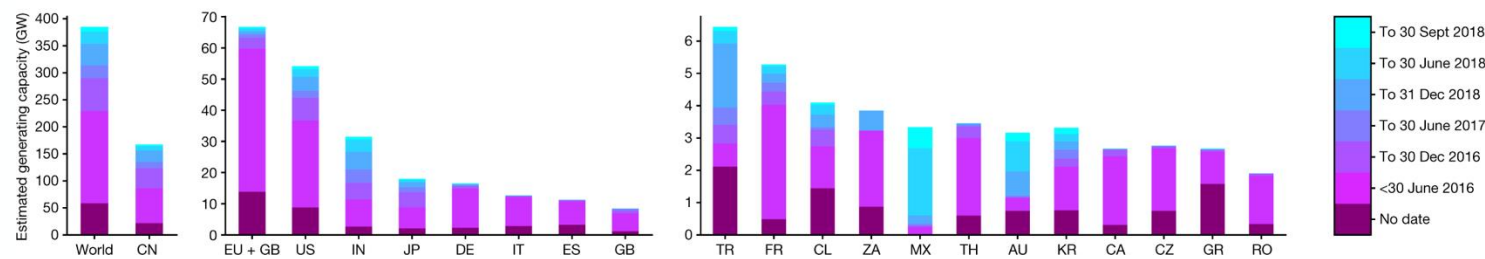
Policy-relevant example



A global inventory of photovoltaic solar energy generating units

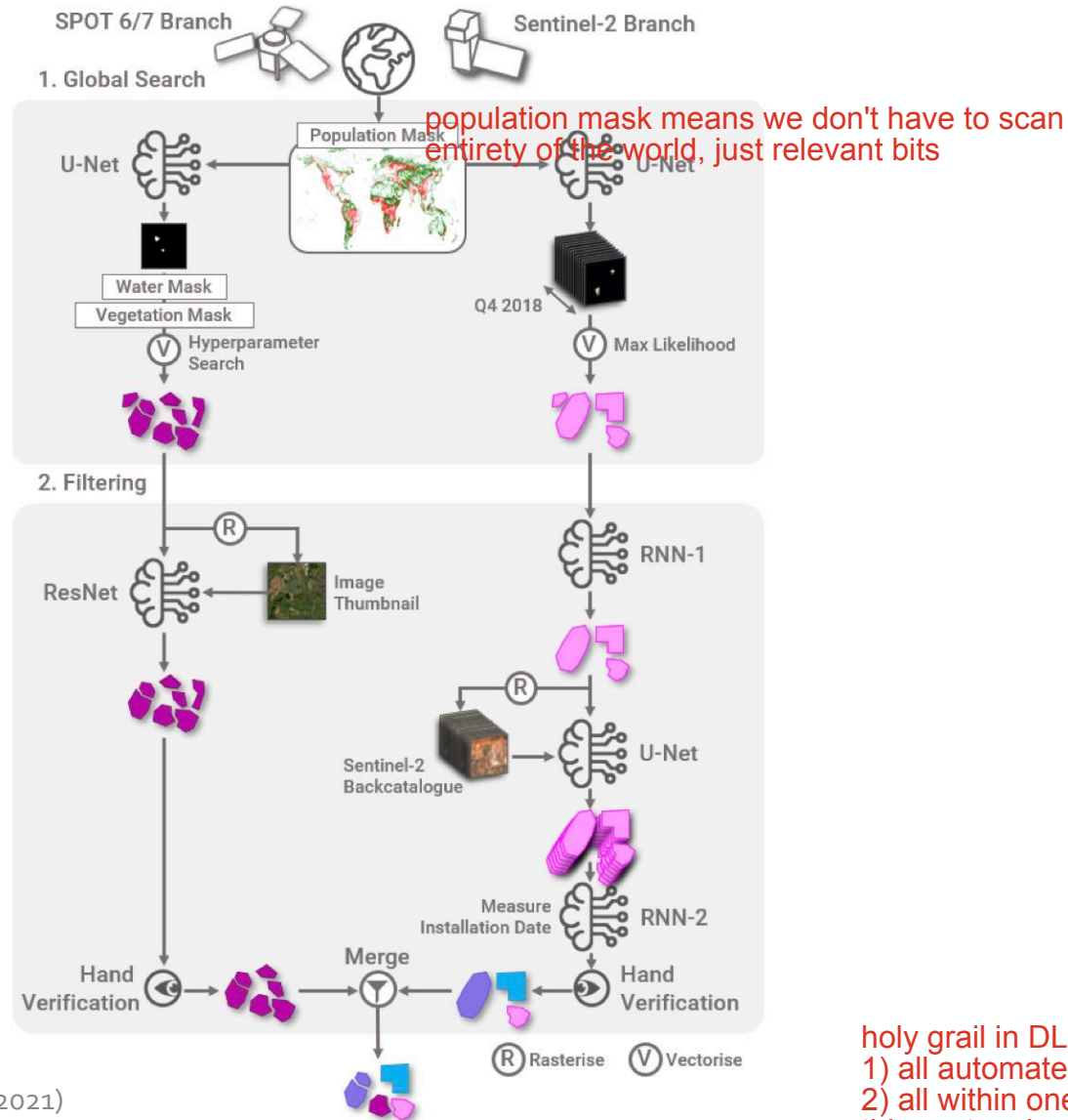
Kruitwagen et al. (2021)

- Detecting commercial, industrial and utility-scale solar PV globally
- Learned information: Location, installation date, capacity
- Segmentation needed for extracting the size and computing the capacity



Policy-relevant example

nb they went for
high recall over precision
- trade off in this case



A global inventory of photovoltaic solar energy generating units

- Sentinel-2: multispectral data (12 channels) by European Space Agency (10m resolution)
- SPOT-6/7: color + infrared (4 channels) by Airbus (1.5 m resolution)
- First maximize pipeline recall, subsequent filtering stages eliminate false positives
- Merging two branches
- Hand verification
- RNN for time series of images

holy grail in DL: End-to-End
1) all automated
2) all within one model
this system is neither - the reality can be different

- Labeled data and augmentation
- Modern CNN models
- Object detection
- Semantic Segmentation

Hertie School
Friedrichstraße 180
10117 Berlin, Germany
T +49 (0)30 259219-0
F +49 (0)30 259219-11
info@hertie-school.org
www.hertie-school.org